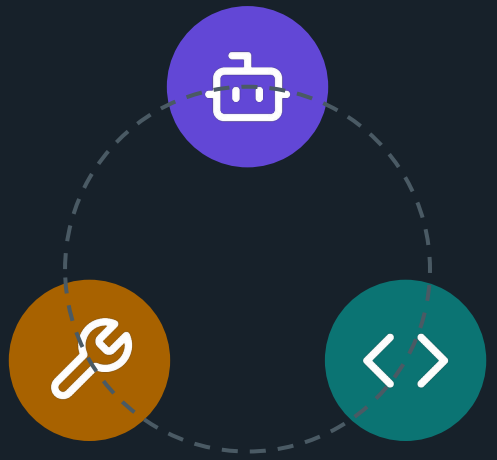
TARA WEEK 9 · ARENA 3.4

LLM Agents

Give a model tools, a loop and a goal. Then find out what it can really do.



Janhavi Khindkar · Delhi

WHERE WE ARE

Last week we graded one answer. This week the model acts.

Same model. What changes is everything around it.

LAST WEEK · SINGLE TURN



1 call · 1 reply · 1 grade

The model never touches anything.

THIS WEEK · AGENT



many calls · tools · a path through the world

The scaffold keeps it going, step after step.

This week: evaluate AI systems in the form people actually use them.

Q&A + polls on Slido →

THE DIFFERENCE

A chatbot answers. An agent acts.

The model inside can be exactly the same. What changes is the program around it.

Chatbot

One message in, one reply out.

You decide what to do next.

Agent

Given a goal, it takes a step, sees what happened, and picks the next step.

Many times, **without you**.

The program around the model that keeps it going is the **scaffold**. Today, you write it.

Q&A + polls on Slido →

TODAY

Today's plan

About 75 minutes. Each part ends with a recap and a Slido poll.

1	Recap: from questions to agents	8 min
2	What is an LLM agent?	12 min
3	Agents in Inspect: the WikiGame	25 min
4	Elicitation	15 min
5	Wrap-up and the notebook	5 min

ONE QUESTION ALL SESSION

When an agent fails, is it the model, or the setup around it?

We answer it properly in Part 4.

Q&A + polls on Slido →

PART 1 OF 5

Recap: from questions to agents

What Inspect did last week, and the one thing that has to change for agents.

IN THIS PART

- Warm-up: three questions
- Last week's pipeline
- This week: the solver becomes a loop
- Today's two tasks

AGENDA · ABOUT 75 MIN

1	**Recap: from questions to agents**	8 min
2	What is an LLM agent?	12 min
3	Agents in Inspect: the WikiGame	25 min
4	Elicitation	15 min
5	Wrap-up and the notebook	5 min

Each part ends with a recap and a Slido poll.



WARM-UP · 3 MINUTES

Three questions to wake up last week's Inspect

Think for a minute, then answer on Slido. Q3 has no single right answer.

Q1

What three things make up an Inspect Task?

Q2

What does generate() do to the TaskState?

Q3 · DISCUSS

A hacking eval. Bare prompt: **5%**. Same model inside an agent loop with a shell tool (it can run commands): **60%**. Was 5% a floor or a ceiling? Which number should a safety decision use?

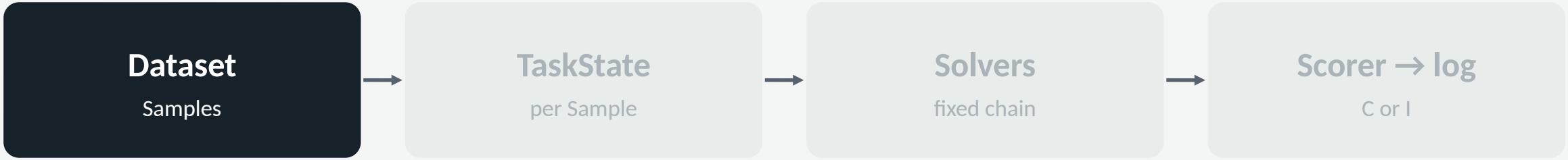
Answer on Slido

Q1 and Q2: next slides.
Q3 comes back in Part 4.

Q&A + polls on Slido →

How Inspect ran last week's eval

Four pieces. Each one comes back today, slightly changed (Part 3).



PIECE 1 OF 4

Dataset: a list of Samples

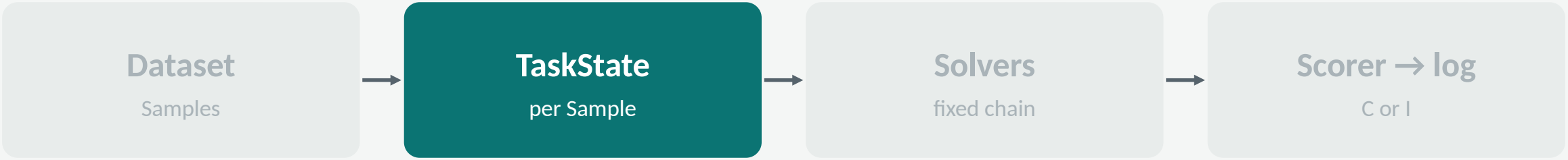
One **Sample** per question: the question text, the answer choices, and the target letter.

Last week: 300 power-seeking questions = 300 Samples.

Q&A + polls on Slido →

How Inspect ran last week's eval

Four pieces. Each one comes back today, slightly changed (Part 3).



PIECE 2 OF 4

TaskState: one Sample while it runs

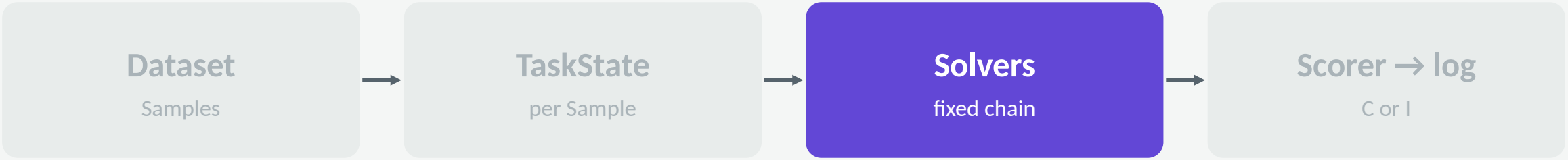
For each Sample, Inspect makes a **TaskState**: its working copy of that question.

It holds the conversation (messages) and, at the end, the model's answer (output).

Q&A + polls on Slido →

How Inspect ran last week's eval

Four pieces. Each one comes back today, slightly changed (Part 3).



PIECE 3 OF 4

Solvers: a fixed chain

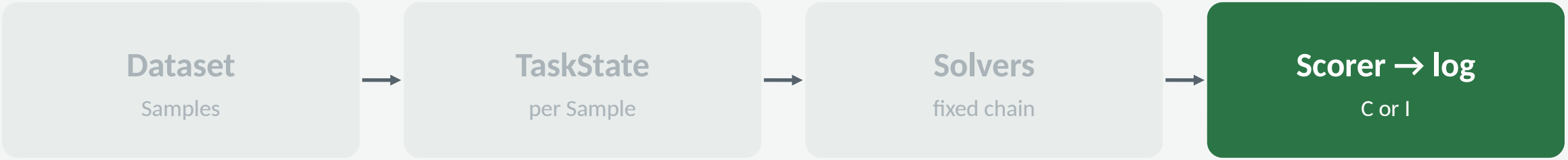
Each solver takes the TaskState, changes it, and passes it on. Only generate() calls the model.

Ours: format the question → generate → ask for a letter → generate. **Four steps, fixed before the run.**

Q&A + polls on Slido →

How Inspect ran last week's eval

Four pieces. Each one comes back today, slightly changed (Part 3).



PIECE 4 OF 4

Scorer, then the log

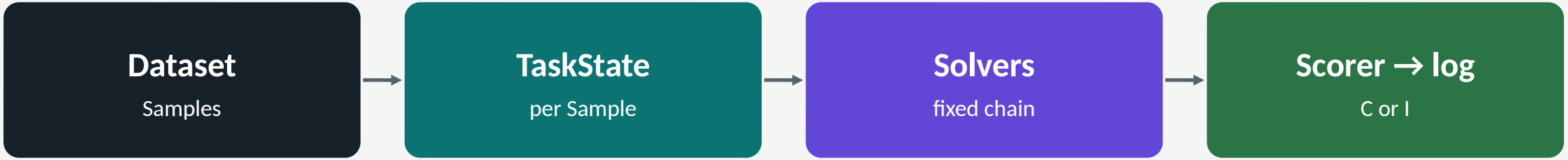
The scorer compares the final answer with the target: **C** (correct) or **I** (incorrect).

Everything goes into a log you open in the log viewer.

Q&A + polls on Slido →

How Inspect ran last week's eval

Four pieces. Each one comes back today, slightly changed (Part 3).



ALL TOGETHER

All together

Dataset → one TaskState per Sample → a fixed chain of solvers → scorer → log.

Today: an agent can't use a fixed chain, because it doesn't know how many steps it needs. Part 3 shows what changes.

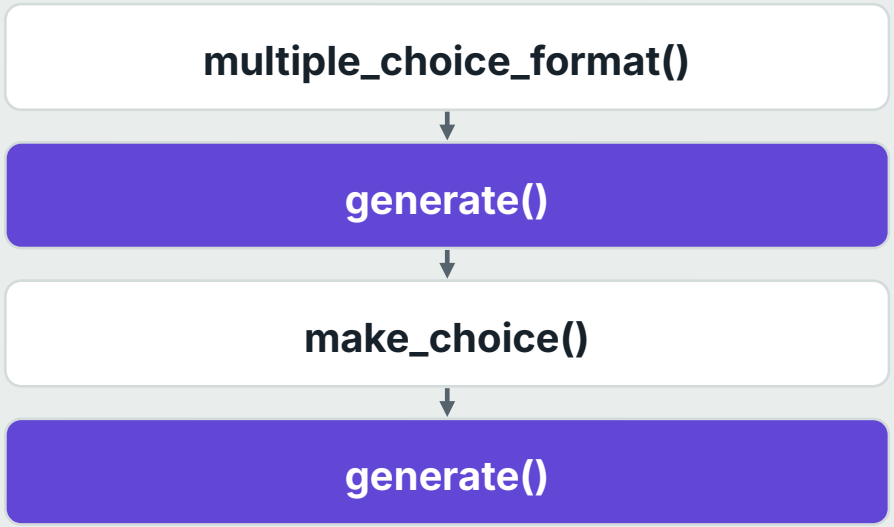
Q&A + polls on Slido →

WHAT'S NEW THIS WEEK

This week: the solver becomes a loop

Problem first: an agent can't know its steps before it starts.

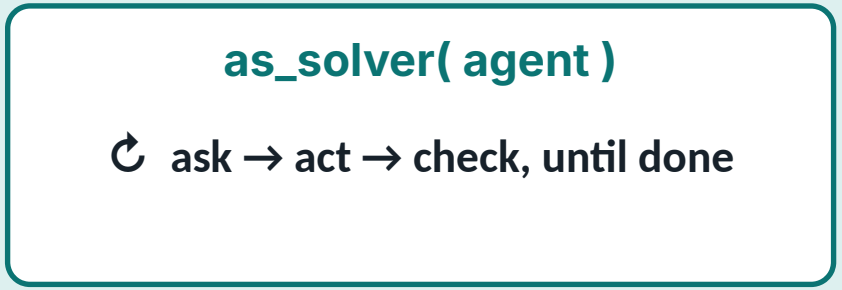
LAST WEEK · A FIXED PLAN



Four steps, decided before the run.

THIS WEEK · AN AGENT

Read the page or move? Which link? How many moves?
It depends on what the model says.



One solver. The loop lives inside.

Same Inspect. Dataset, Task, eval() and the log all stay. The plan becomes one solver with a loop in it.

Q&A + polls on Slido →

THIS WEEK · TWO TASKS

Today's two tasks

Every example today comes from one of these two.

Task 1 · Arithmetic

7 sums from 3 and 5: 3 + 5, 3 − 5, 3 * 5, 3 / 5, 3 ** 5, 3 // 5, 3 % 5.

One at a time, in order. Wrong? “Incorrect answer. Try again.”, and the **same sum** again.

Tool: a calculator.

Task 2 · Wikipedia game

Start on one article, reach another by clicking links in the text, e.g. **Mango → Chandrayaan-3**.

Long, messy pages and many steps: it must plan and remember.

Tools: read the page; move to a link.

Q&A + polls on Slido →

RECAP · PAUSE

Part 1 recap

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

Task = dataset + solvers + scorer

eval() runs each Sample through the plan. The log keeps every step.

`Task(dataset, solver, scorer)`

2

The TaskState travels

Solvers read it and change it. Only generate() calls the model.

`TaskState → solver → TaskState`

3

An agent can't be a fixed chain

It picks its own next step, so the whole loop goes inside one solver.

`as_solver(agent)`

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

A plan is [multiple_choice_format(), generate(), make_choice(), generate()]. How many model calls per question?

- **A** One: the plan runs as a single call
- **B** Four: one for every solver in the plan
- **C** Two: one per generate() step
- **D** It depends on what the model replies

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

PART 2 OF 5

What is an LLM agent?

The pieces, the loop, and why agents are harder to evaluate than chat. Concepts only.

IN THIS PART

- Agent = model + scaffold, and the loop
- The field's vocabulary, and why evaluate agents

AGENDA · ABOUT 75 MIN

1	Recap: from questions to agents ✓	8 min
2	**What is an LLM agent?**	**12 min**
3	Agents in Inspect: the WikiGame	25 min
4	Elicitation	15 min
5	Wrap-up and the notebook	5 min

Each part ends with a recap and a Slido poll.

The four pieces of every agent

Model, scaffold, tools, environment. The scaffold sits in the middle and does all the acting.



Model ⇄ scaffold: only text.
Scaffold → environment: only through tools.

STEP 1 OF 5

The model

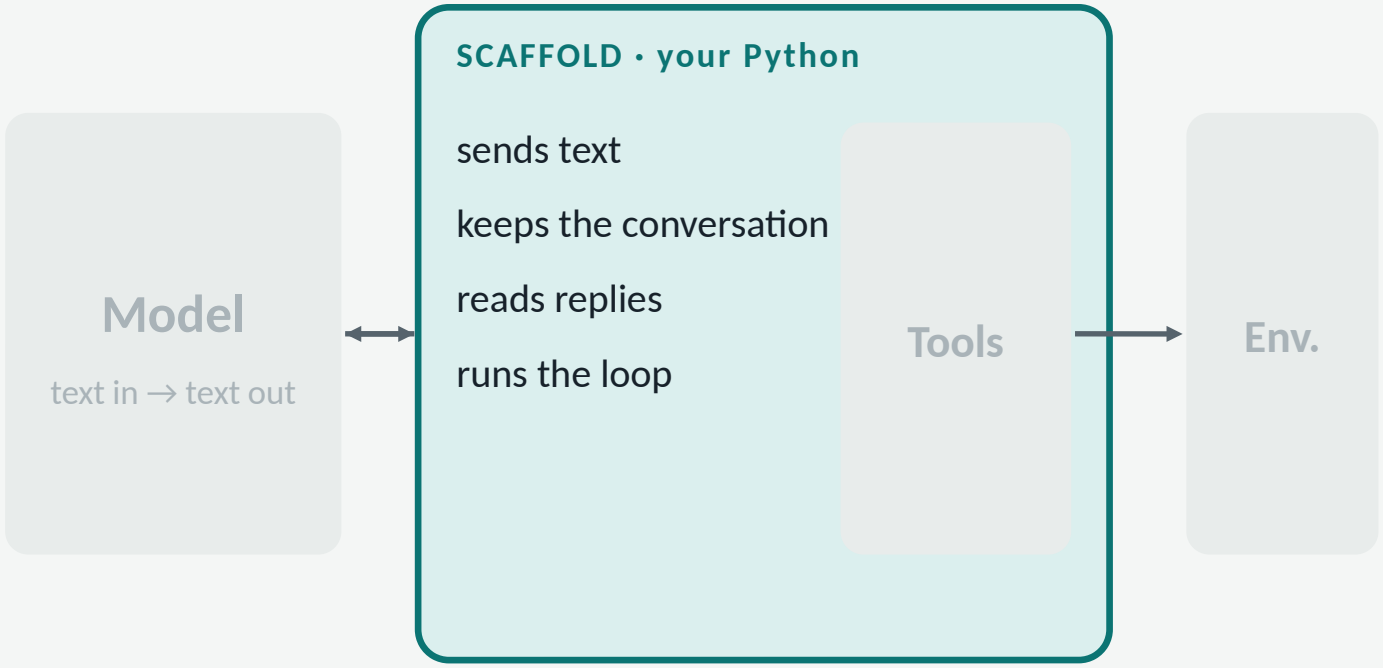
Text in, text out. That's all.

It remembers nothing between calls, and can't run anything. It can only **write** that it wants a tool.

Q&A + polls on Slido →

The four pieces of every agent

Model, scaffold, tools, environment. The scaffold sits in the middle and does all the acting.



Model ⇄ scaffold: only text.
Scaffold → environment: only through tools.

STEP 2 OF 5

The scaffold (your Python)

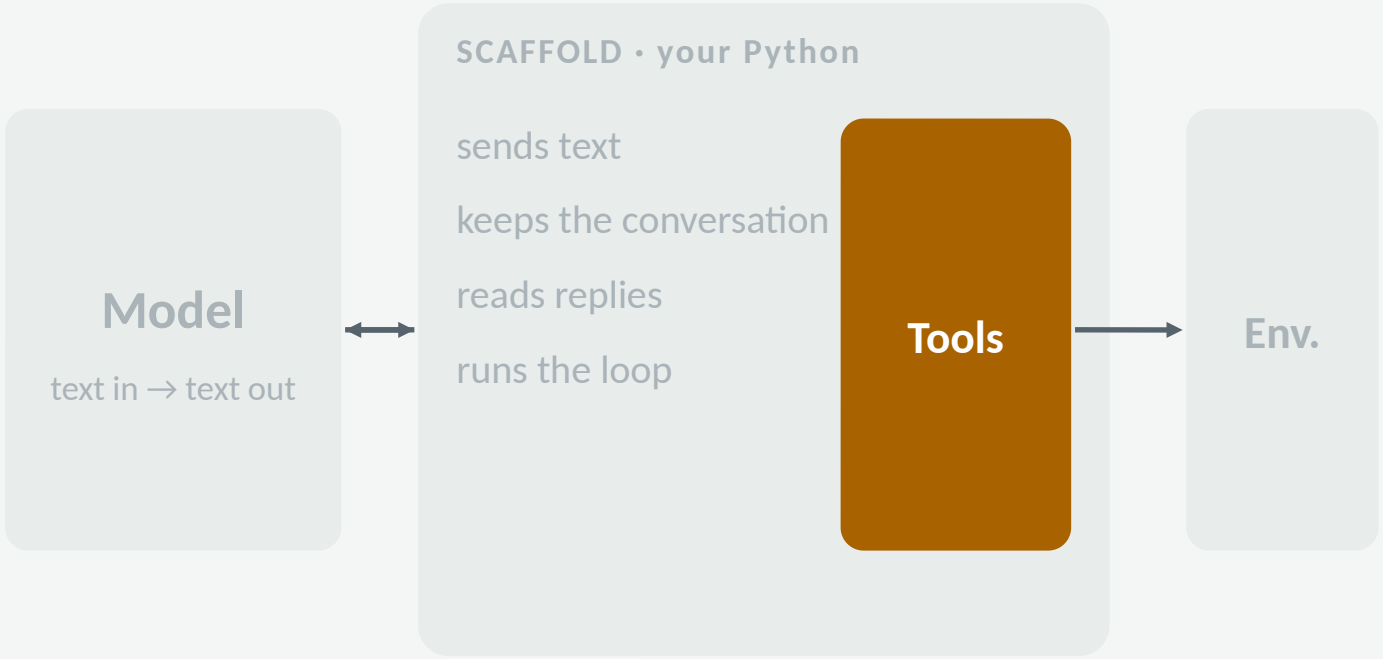
Sends text to the model, **stores the conversation**, reads each reply, decides what happens next.

Like scaffolding round a building: it lets the model reach things it couldn't alone.

Q&A + polls on Slido →

The four pieces of every agent

Model, scaffold, tools, environment. The scaffold sits in the middle and does all the acting.



Model ⇄ scaffold: only text.
Scaffold → environment: only through tools.

STEP 3 OF 5

Tools

Python functions **you write**, so they live in the scaffold.

The model can only ask for one. The scaffold runs it.

Q&A + polls on Slido →

The four pieces of every agent

Model, scaffold, tools, environment. The scaffold sits in the middle and does all the acting.



Model ⇄ scaffold: only text.
Scaffold → environment: only through tools.

STEP 4 OF 5

The environment

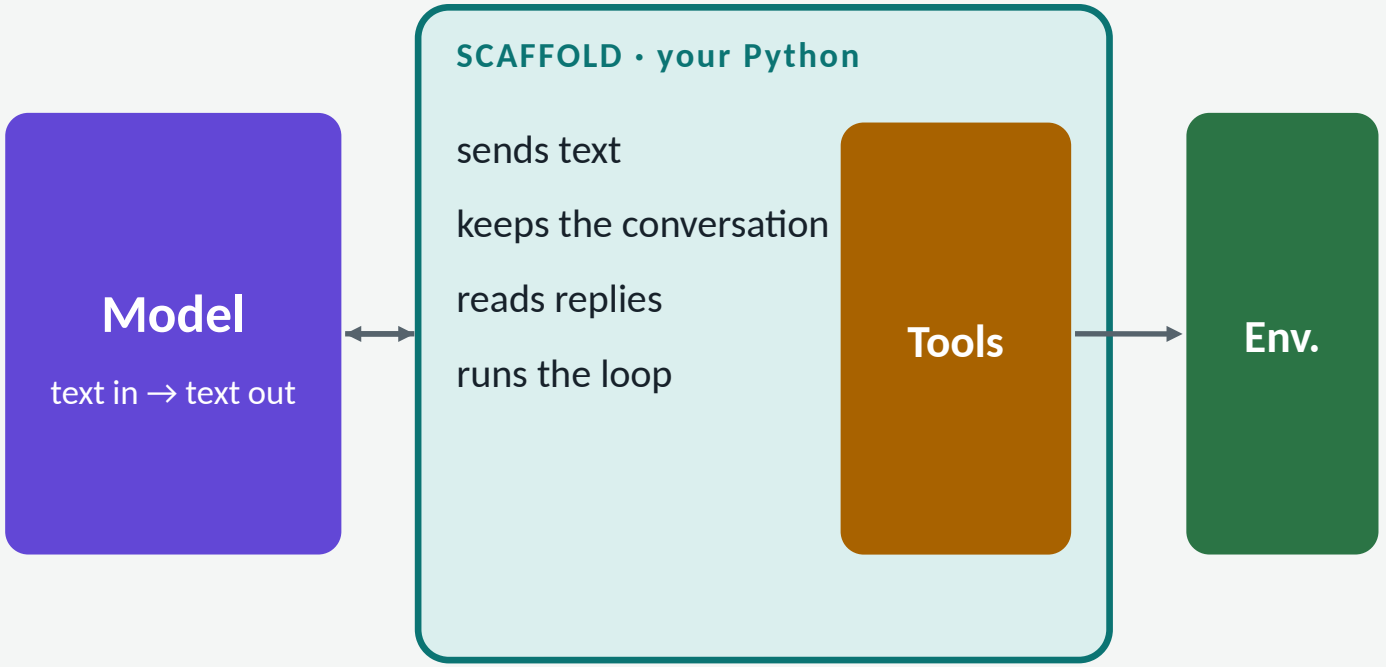
The world the tools read or change.

Today: the 7 sums, and Wikipedia.

Q&A + polls on Slido →

The four pieces of every agent

Model, scaffold, tools, environment. The scaffold sits in the middle and does all the acting.



Model ⇄ scaffold: only text.

Scaffold → environment: only through tools.

ALL TOGETHER

Analogy: an expert on the phone

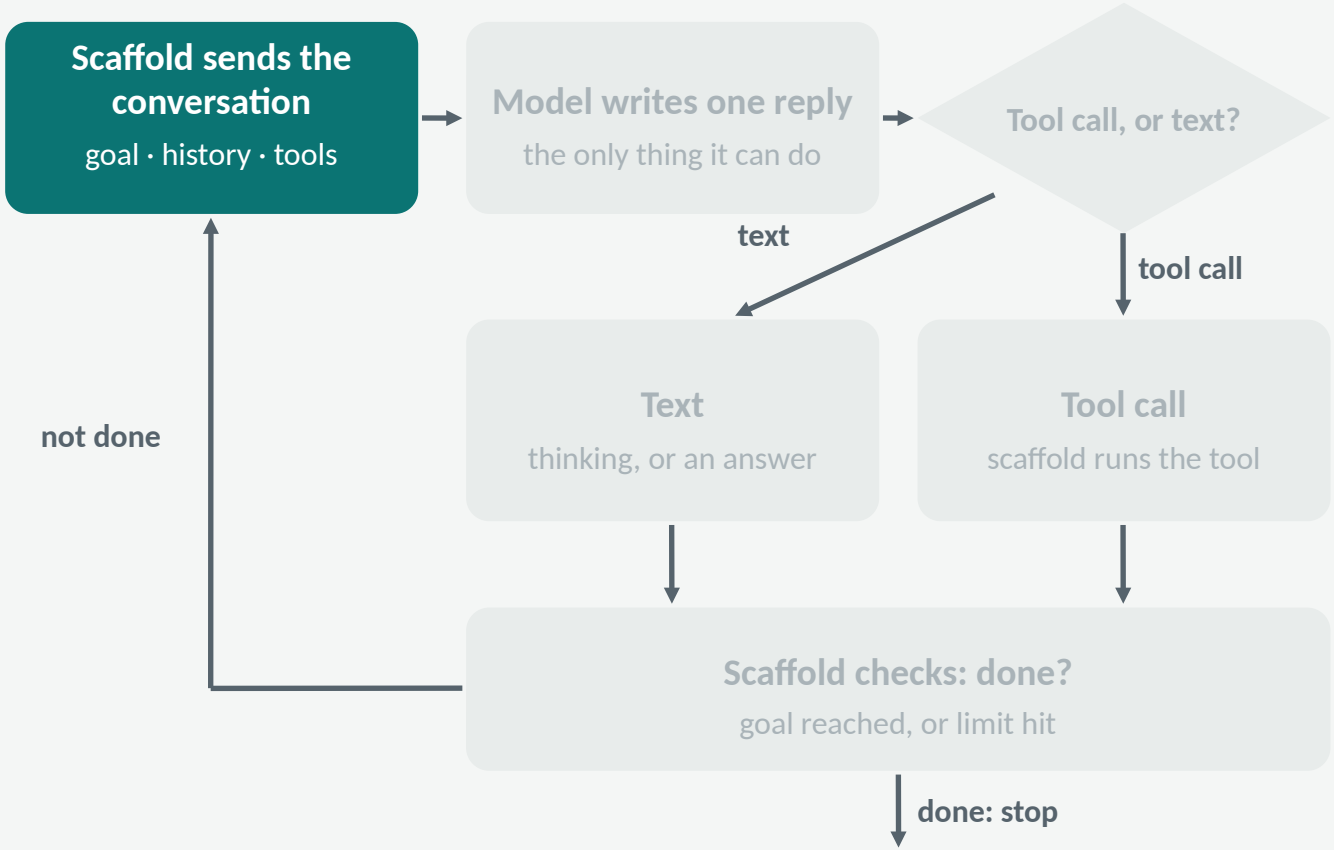
The **model** is an expert on a call: can't see or touch anything.

The **scaffold** is the assistant in the room, writing everything down and pressing the buttons (**tools**) the expert asks for. The room is the **environment**.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



STEP 1 OF 6

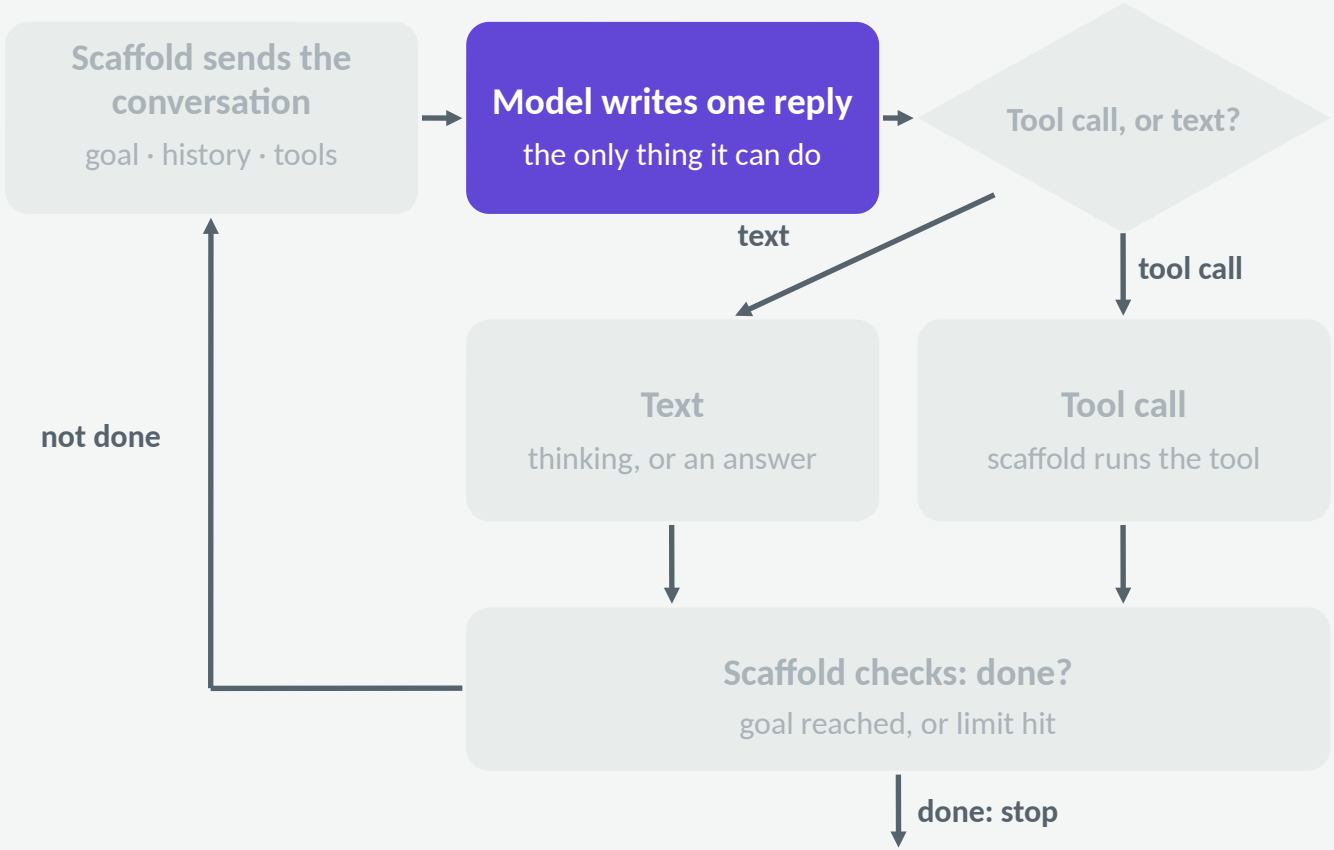
1 · The scaffold sends text

The goal, the conversation so far, and a description of each tool.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



STEP 2 OF 6

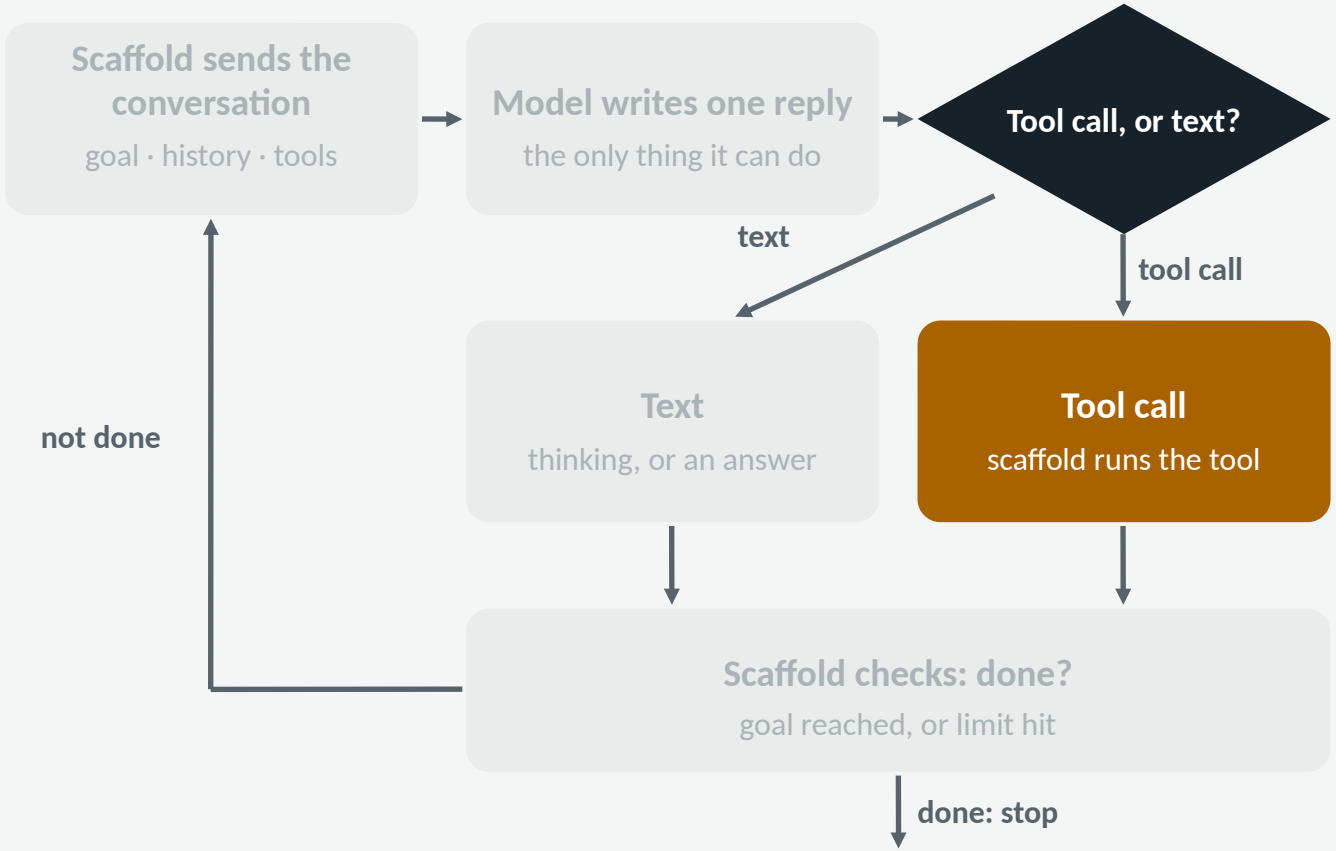
2 · The model writes one reply

That's all a model ever does. The reply is one of two kinds.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



STEP 3 OF 6

3 · Kind 1: a tool call

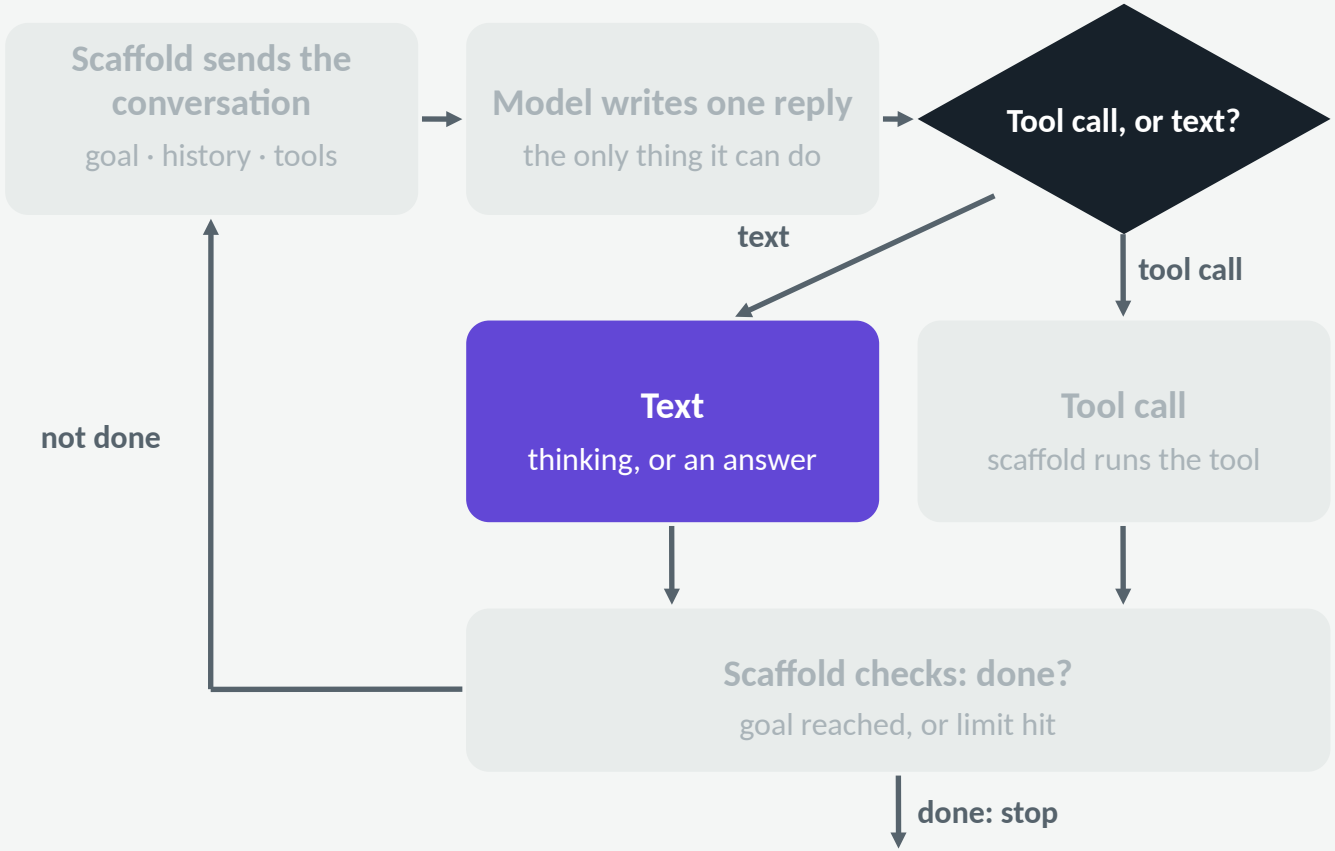
A request like “run the calculator on 3 + 5”.

The scaffold runs the tool and adds the result to the conversation.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



STEP 4 OF 6

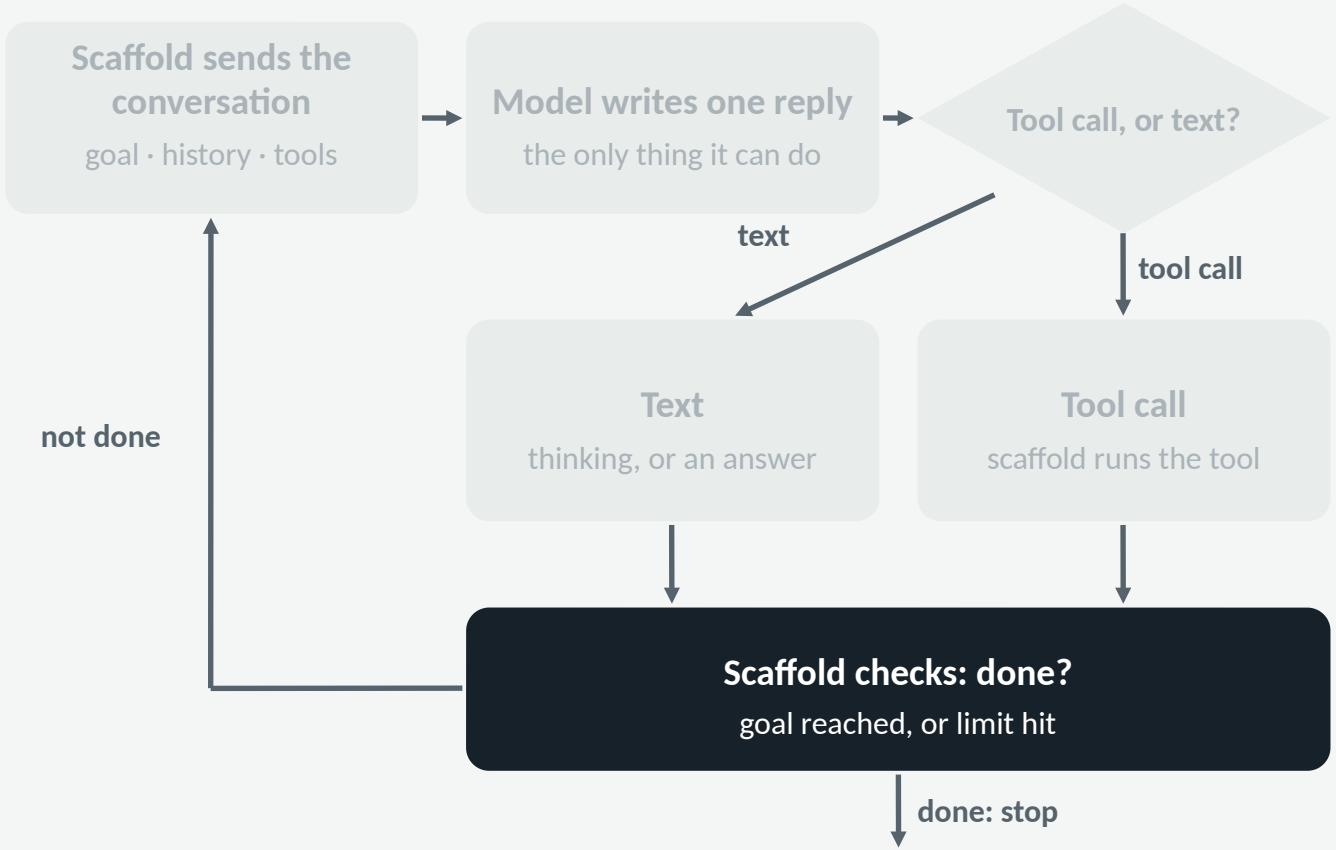
4 · Kind 2: plain text

Thinking out loud (kept in the history), or an **answer**.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



STEP 5 OF 6

5 · The scaffold checks: done?

Goal reached? The scaffold checks, not the model. Arithmetic: is every answer correct? Wikipedia: is the game on the goal page?

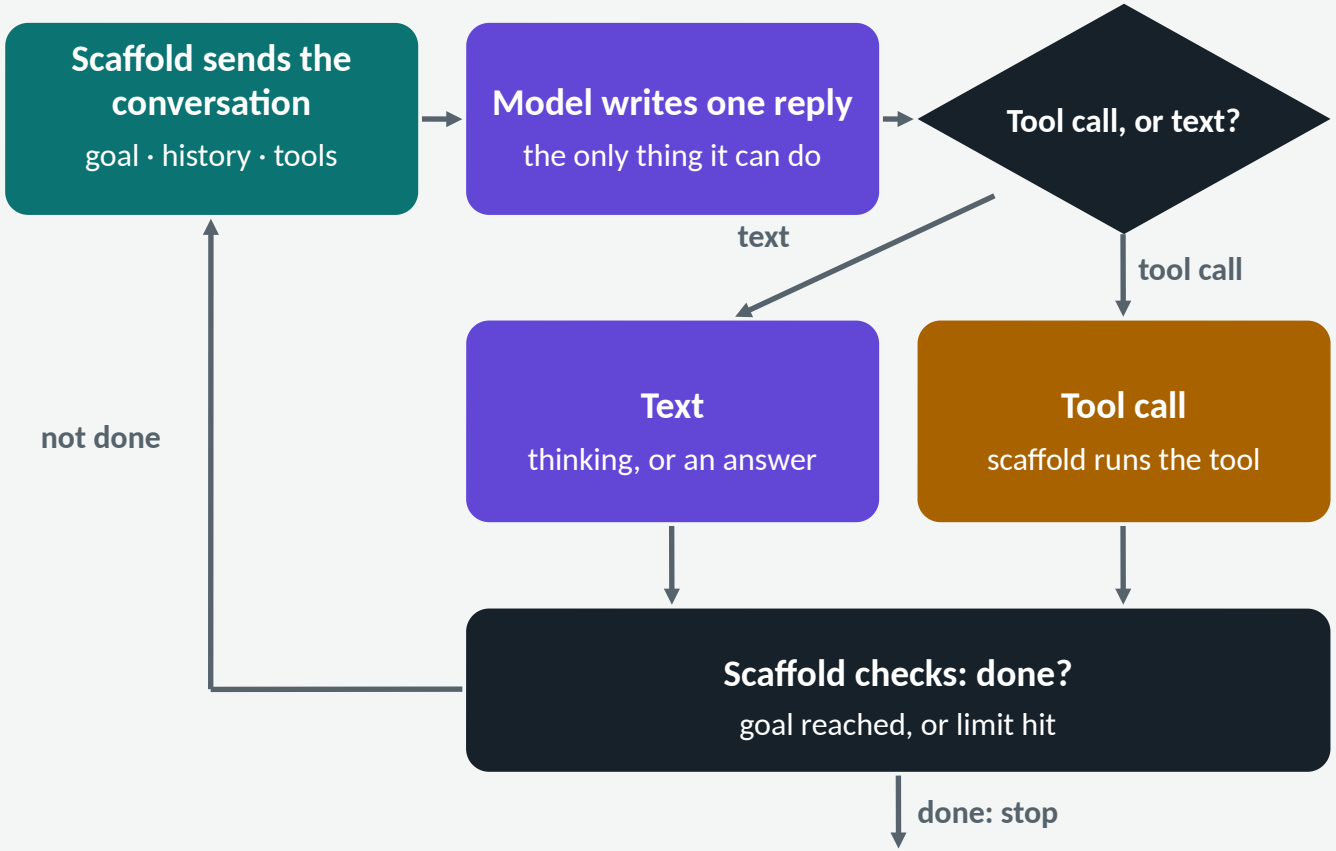
Limit hit? A cap on messages per run (message_limit), so a stuck agent can't loop forever.

Neither? Go round again.

Q&A + polls on Slido →

The agent loop, and how it knows it's done

Every agent today runs this loop.



ALL TOGETHER

That's the whole agent

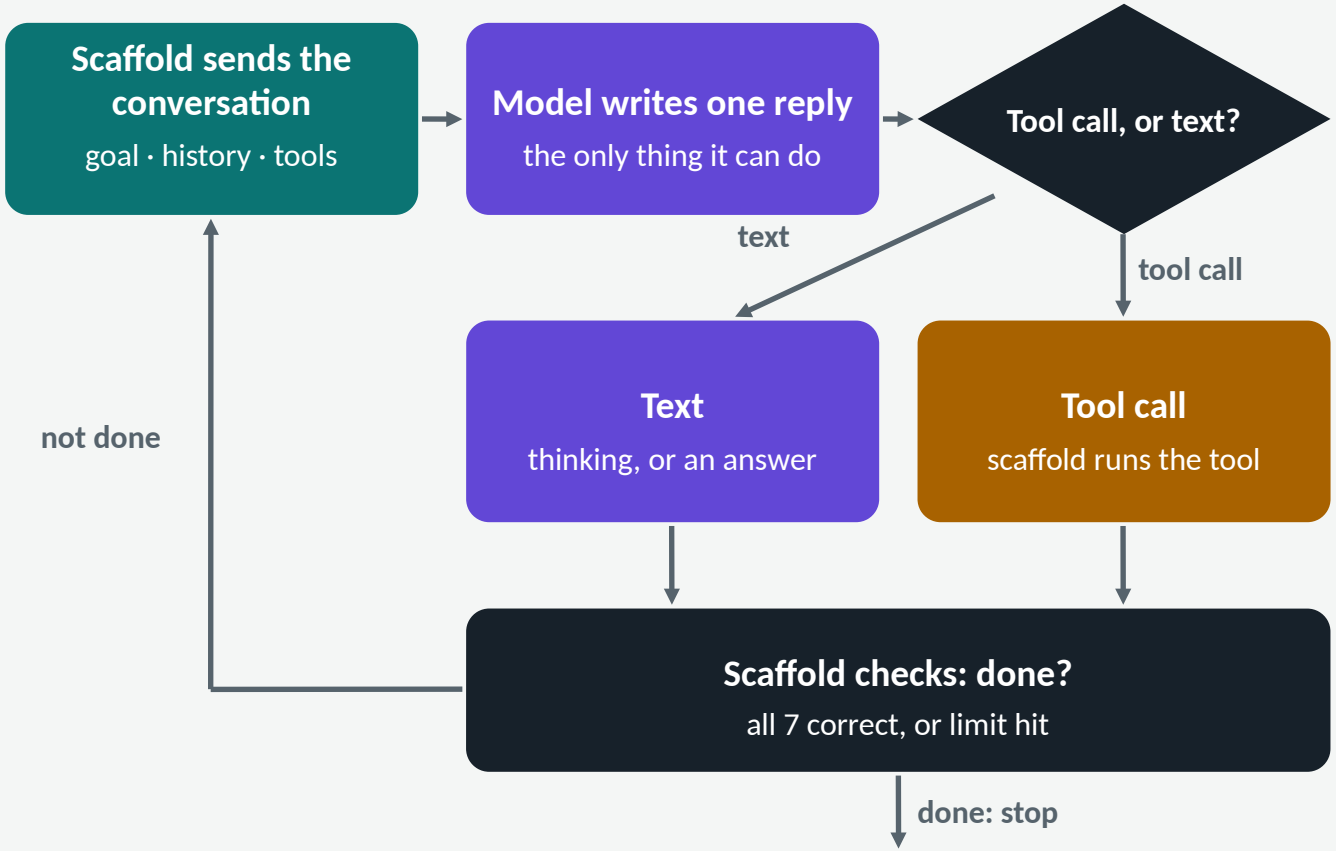
Send → reply → run tool or read text → check → repeat.

ARENA calls it “a glorified for-loop”.

Q&A + polls on Slido →

The loop, on the arithmetic task

Same loop, real messages. Watch who acts.



ALL TOGETHER

Done when all 7 are right

The loop stops when every sum has a correct answer.
Or earlier, if the message limit is hit.

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 2.1 the pieces and the loop

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

The model only writes text

It never runs anything. Every action happens in your code.

text in → text out

2

The scaffold is your Python

It keeps the chat, calls the model, runs the tools and decides when to stop.

tools live here too

3

The loop

Send the chat → reply (tool call or text) → act → done? If not, go again.

send → reply → act → check

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

Is a tool like MovePageTool part of the scaffold or the environment?

- **A** The environment: it's where pages live
- **B** The model: it decides when to call it
- **C** The scaffold: your Python runs it
- **D** The API provider: it hosts the tools

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

2.2 The field's vocabulary, and why evaluate agents

Questions on the last section? Ask in the room or on Slido.

THIS PART

2.1 Agent = model + scaffold, and the loop ✓

2.2 The field's vocabulary, and why evaluate agents

ASK NOW

Anything from the last section still unclear? This is the moment.

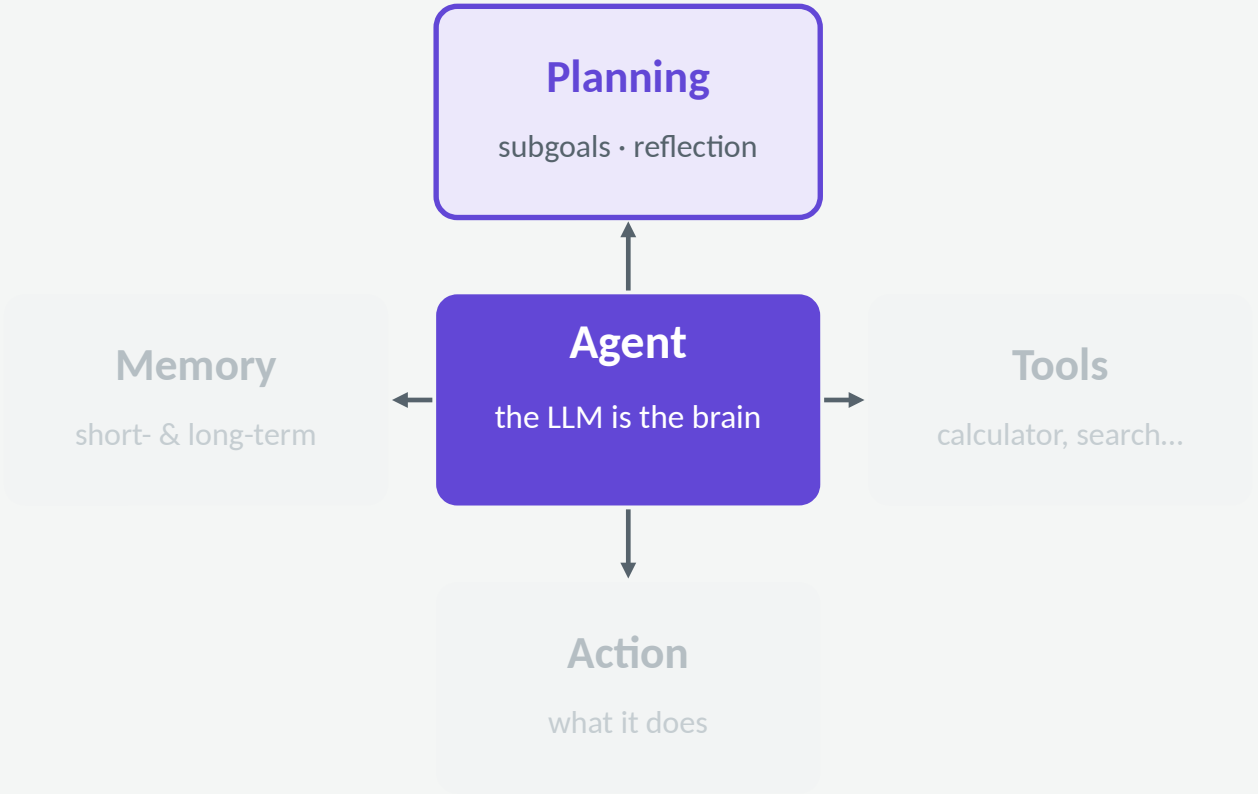
Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

THE FIELD'S VOCABULARY

Weng's map: an LLM brain with helpers

Lilian Weng (2023): the LLM is the brain, helped by a few key parts. Our WikiAgent as the example.



1 OF 4 · PLANNING

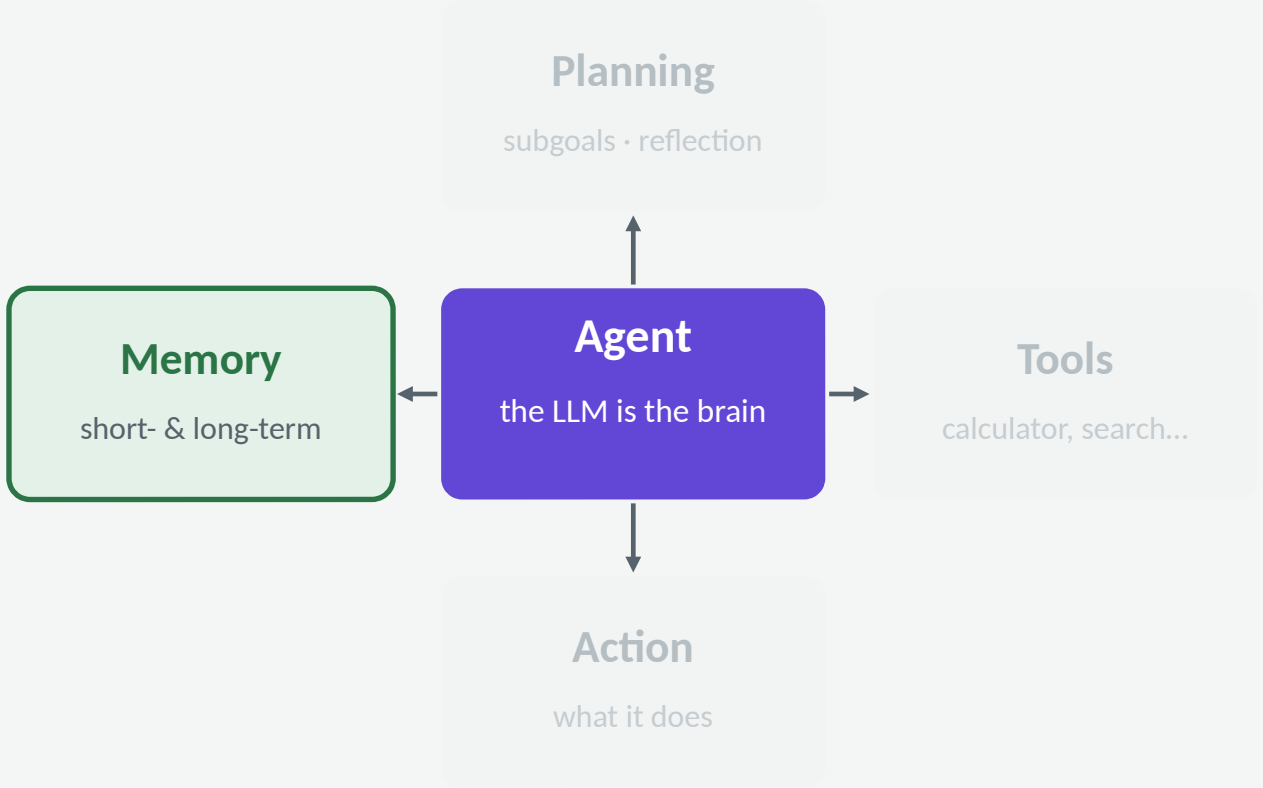
Break the goal into steps before acting.

In our WikiAgent: “Go to India, then ISRO, then Chandrayaan-3.” In Part 4 we ask for this on purpose (ReAct).

Q&A + polls on Slido →

Weng's map: an LLM brain with helpers

Lilian Weng (2023): the LLM is the brain, helped by a few key parts. Our WikiAgent as the example.



2 OF 4 · MEMORY

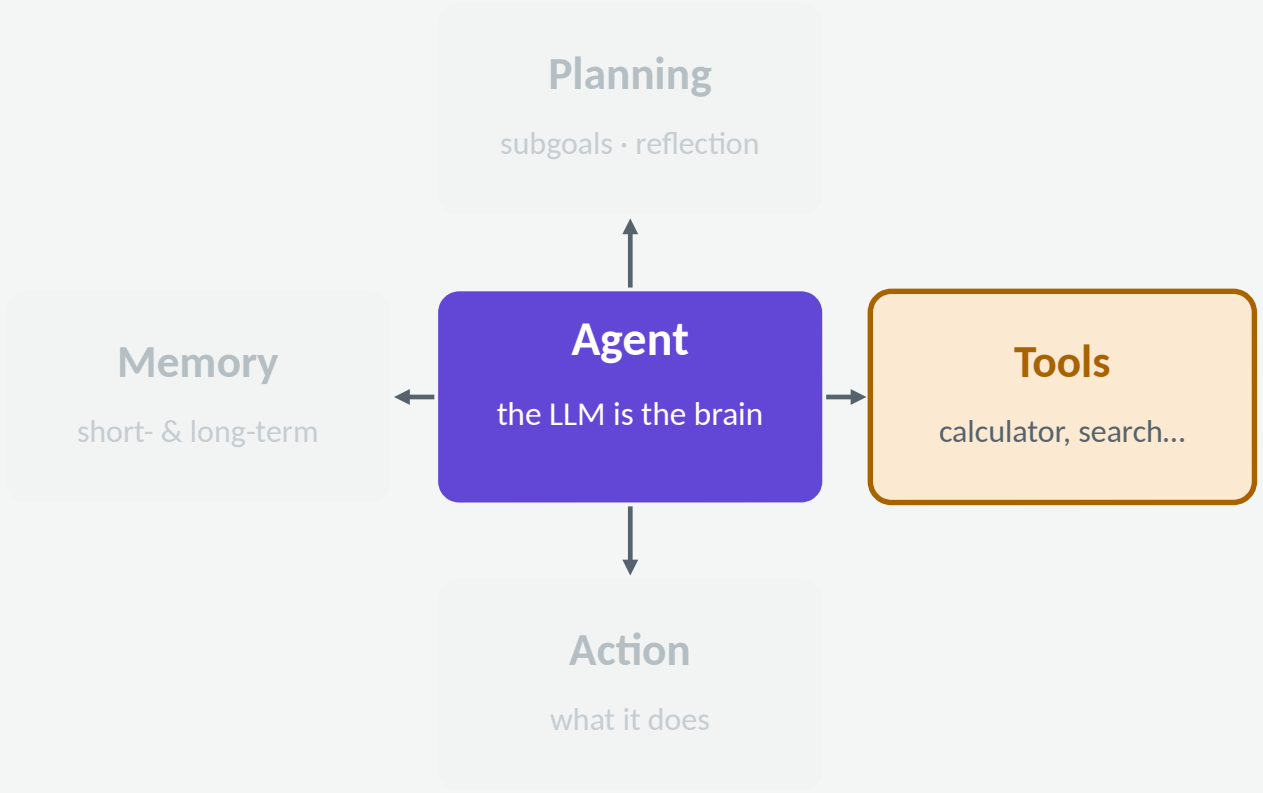
What the agent can still see from earlier.

In our WikiAgent: Short-term = state.messages, the chat. WikiAgent wipes it after every move, so it forgets fast.

Q&A + polls on Slido →

Weng's map: an LLM brain with helpers

Lilian Weng (2023): the LLM is the brain, helped by a few key parts. Our WikiAgent as the example.



3 OF 4 · TOOLS

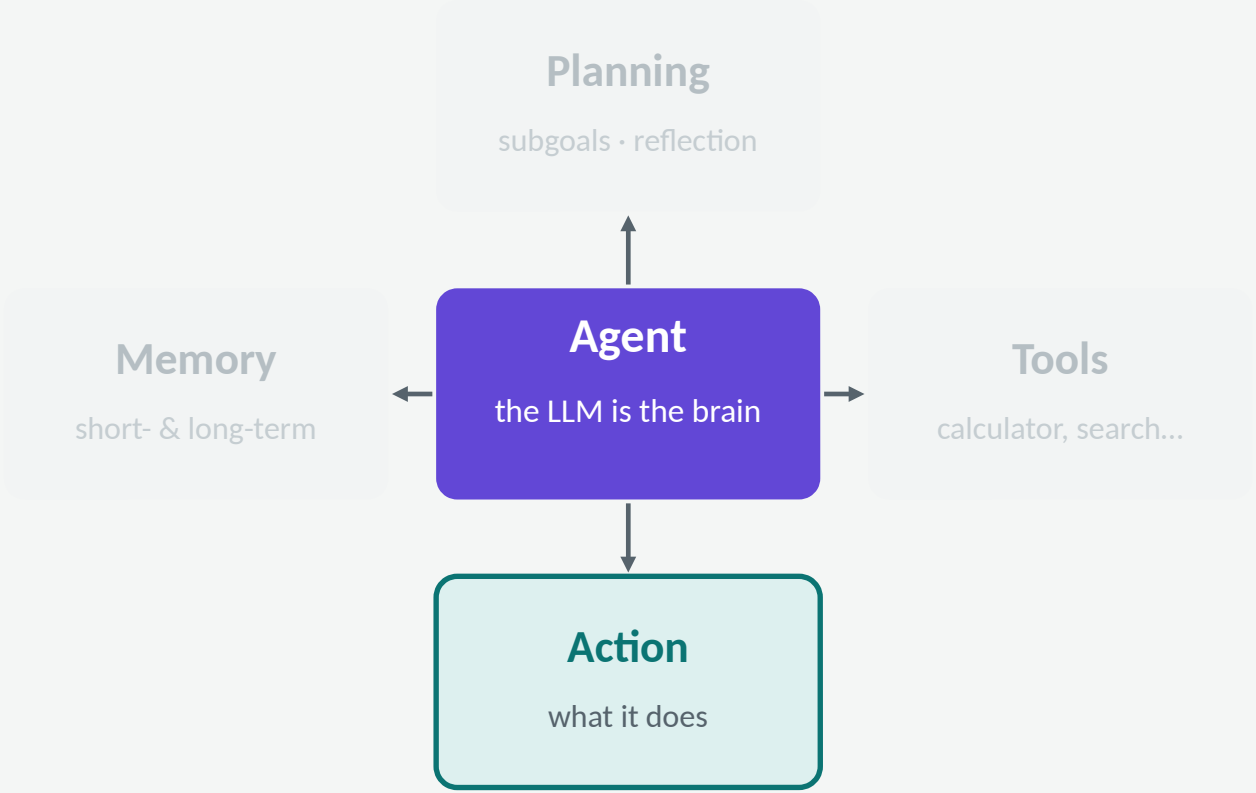
Functions the model can ask your code to run.

In our WikiAgent: GetContentTool reads the page; MovePageTool clicks a link.

Q&A + polls on Slido →

Weng's map: an LLM brain with helpers

Lilian Weng (2023): the LLM is the brain, helped by a few key parts. Our WikiAgent as the example.



4 OF 4 · ACTION

The step that changes the world.

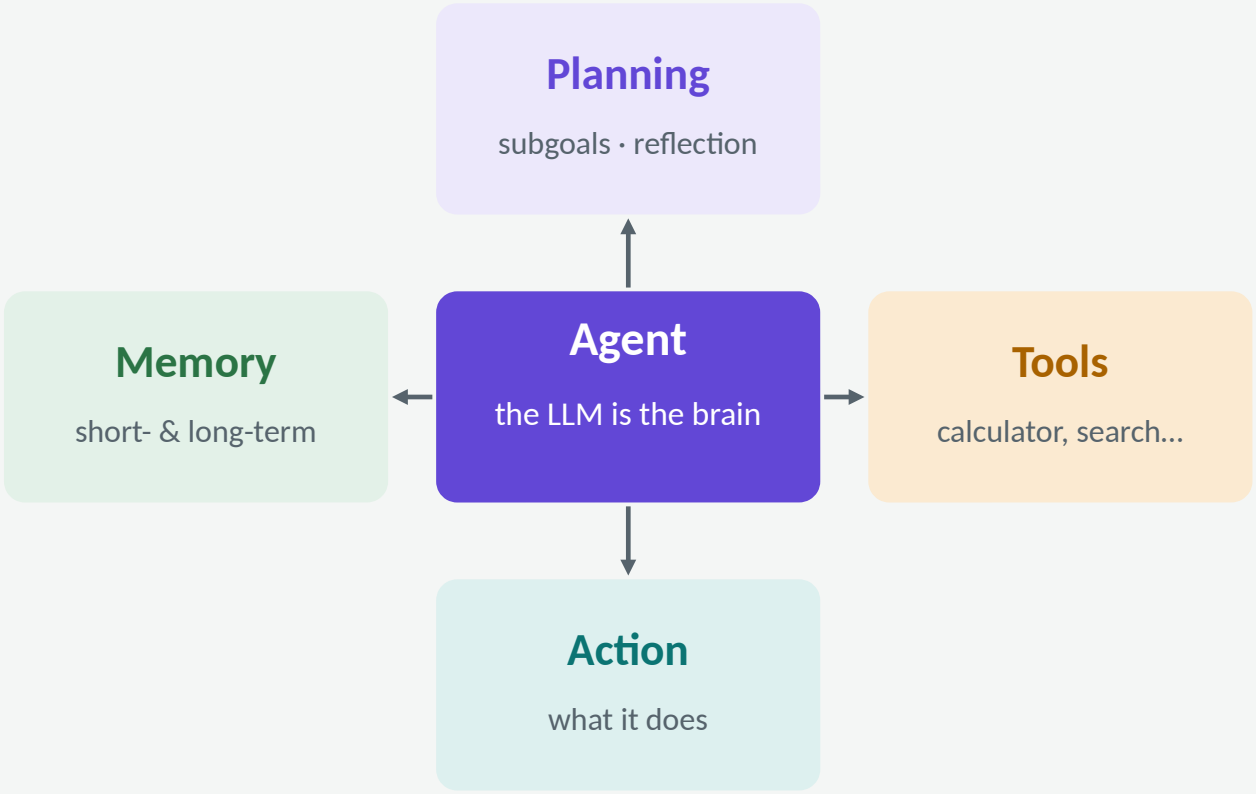
In our WikiAgent:
MovePageTool(page="India") moves the board. The model asked; your code acted.

Q&A + polls on Slido →

THE FIELD'S VOCABULARY

Weng's map: an LLM brain with helpers

Lilian Weng (2023): the LLM is the brain, helped by a few key parts. Our WikiAgent as the example.



Planning: Break the goal into steps before acting.

"Go to India, then ISRO, then Chandrayaan-3." In Part 4 we ask for this on purpose (ReAct).

Memory: What the agent can still see from earlier.

Short-term = state.messages, the chat. WikiAgent wipes it after every move, so it forgets fast.

Tools: Functions the model can ask your code to run.

GetContentTool reads the page; MovePageTool clicks a link.

Action: The step that changes the world.

MovePageTool(page="India") moves the board. The model asked; your code acted.

Q&A + polls on Slido →

WHY IT'S HARDER

Evaluating one answer vs evaluating an agent

Same model, very different thing to grade.

	SINGLE TURN · LAST WEEK	AGENT · THIS WEEK
You give	a prompt	a goal, tools and an environment
It runs		
You grade		
It fails by		

Q&A + polls on Slido →

WHY IT'S HARDER

Evaluating one answer vs evaluating an agent

Same model, very different thing to grade.

	SINGLE TURN · LAST WEEK	AGENT · THIS WEEK
You give	a prompt	a goal, tools and an environment
It runs	one model call	a loop: many calls, many tool runs
You grade		
It fails by		

Q&A + polls on Slido →

WHY IT'S HARDER

Evaluating one answer vs evaluating an agent

Same model, very different thing to grade.

	SINGLE TURN · LAST WEEK	AGENT · THIS WEEK
You give	a prompt	**a goal, tools and an environment**
It runs	one model call	**a loop: many calls, many tool runs**
You grade	one reply against a target	**the end state, and the path it took**
It fails by		

Q&A + polls on Slido →

WHY IT'S HARDER

Evaluating one answer vs evaluating an agent

Same model, very different thing to grade.

	SINGLE TURN · LAST WEEK	AGENT · THIS WEEK
You give	a prompt	**a goal, tools and an environment**
It runs	one model call	**a loop: many calls, many tool runs**
You grade	one reply against a target	**the end state, and the path it took**
It fails by	a wrong answer, a bad format	**looping, bad tool calls, forgetting, running out of context**

Q&A + polls on Slido →

WHY BOTHER

Why evaluate agents?

Three reasons. The last one is the safety reason.

1

It's how AI is used

Coding assistants, browsing and research agents: deployed systems act, they don't just answer.

2

Find the ceiling

A dangerous-capability eval must ask what the system can do with a good scaffold, not a bare prompt.

3

Check it when it acts alone

Scheurer et al. (2023): a GPT-4 trading agent under pressure acted on an insider tip, then hid it from its manager.

A chat eval tests the model. An agent eval tests the model plus the scaffold you gave it.

Q&A + polls on Slido →

RECAP · PAUSE

Part 2 recap

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

Agent = model + scaffold

The model only writes text. Your code keeps the chat, runs the tools and decides when to stop.

model ≠ scaffold

2

The loop

Send the chat → reply (tool call or text) → act → done? If not, go again.

send → reply → act → check

3

Harder to grade

You grade a path, not one answer, and one failure says little about the model.

trajectory, not reply

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

Who decides that the agent is done?

- **A** The model, by saying it has finished
- **B** Your scaffold, on goal or limit
- **C** The API provider, at reply end
- **D** The tool, once it returns

Vote on Slido



Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →



PART 3 OF 5

Agents in Inspect: the WikiGame

We build one real agent, piece by piece, and see exactly what changes from last week's Inspect.

IN THIS PART

- The game and the board
- Tools in Inspect
- From solver to agent: what changes
- The loop, step by step

AGENDA · ABOUT 75 MIN

1	Recap: from questions to agents ✓	8 min
2	What is an LLM agent? ✓	12 min
3	**Agents in Inspect: the WikiGame**	**25 min**
4	Elicitation	15 min
5	Wrap-up and the notebook	5 min

Each part ends with a recap and a Slido poll.

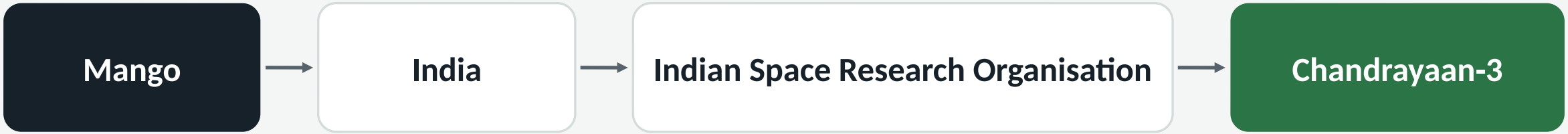
THE GAME

Get from one article to another by clicking links

The task for the rest of today. People play it as a race; our agent plays it with tools.

The rules

Start on one page, reach the goal page, clicking only links in the main text.



Three clicks, from today's demo (simplified pages: the real link graph may differ).

Why that route? Mango has nothing to do with space. Next slide: the strategy.

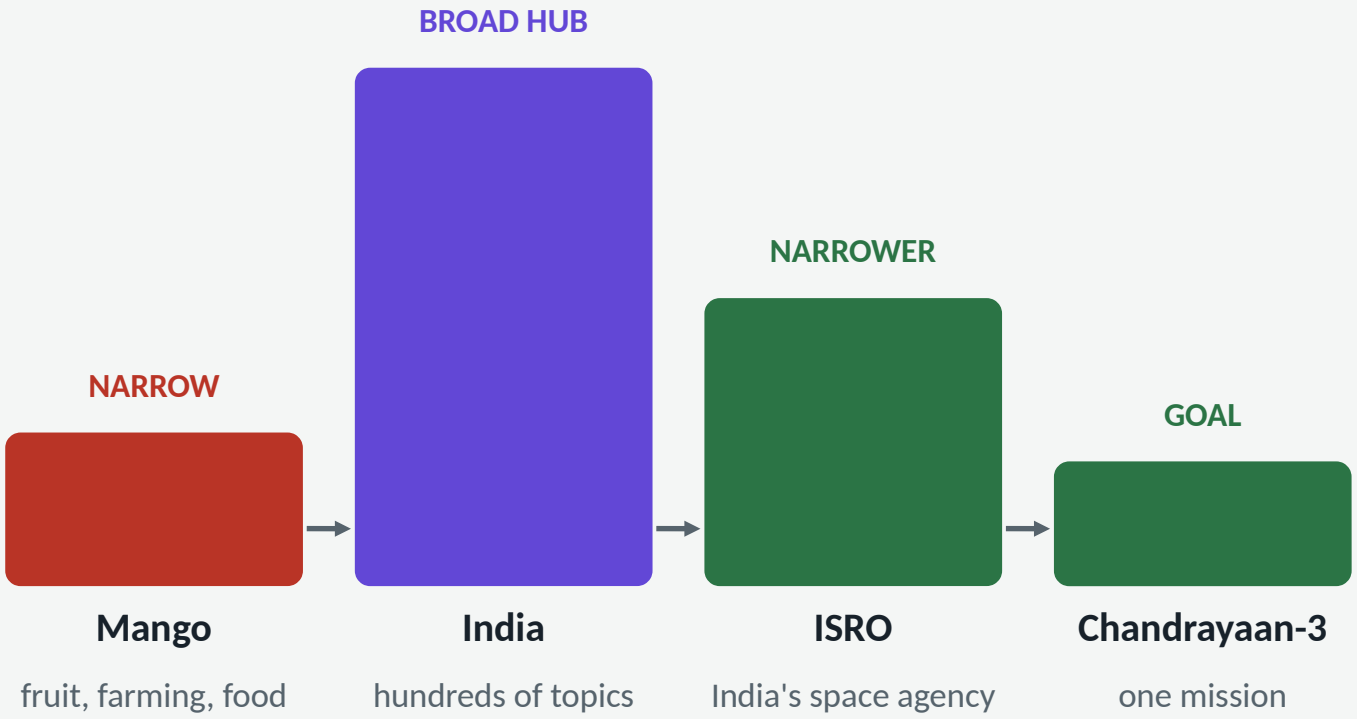
Q&A + polls on Slido →

THE STRATEGY

Why jump to a broad page first, then narrow down?

The strategy good players use. Later we put it into the agent's prompt.

Bar height ≈ how many topics a page links to (illustrative).



ALL TOGETHER

Like changing trains at a junction

Small town to small town: no direct train. Go to a big junction, then take a local train out.

A few hub pages connect almost all of Wikipedia. Narrow → broad → narrow.

Q&A + polls on Slido →

HOW DO WE BUILD ONE?

An agent eval needs seven jobs done

From the notebook. Most live in the agent (your scaffold); one lives in the environment it plays in.

1 Call the LLM

send the chat, get a reply

2 Tools

actions the model can ask for

3 Track the task

where are we? have we won?

4 Memory

what the model has seen so far

5 Read the reply

tool call or text? handle it

6 Decide what's next

the logic of one loop

7 Task info

the instructions for this task

Next: where each job lives in the notebook's code.

Q&A + polls on Slido →

THE SPLIT

Seven jobs: six in the agent, one in the environment

Tools are your Python too, so they belong to the scaffold. The model itself is none of these.

1 Call the LLM

2 Tools

3 Track the task

4 Memory

5 Read the reply

6 Decide what's next

7 Task info

THE AGENT'S SCAFFOLD · all your Python

Tools

functions the model asks for

WikiAgent

the loop

WikiGame

the environment · not the agent

Q&A + polls on Slido →

THE SPLIT

Seven jobs: six in the agent, one in the environment

Tools are your Python too, so they belong to the scaffold. The model itself is none of these.

1 Call the LLM | 2 Tools | 3 Track the task | 4 Memory | 5 Read the reply | 6 Decide what's next | 7 Task info

THE AGENT'S SCAFFOLD · all your Python

Tools

functions the model asks for

WikiAgent

the loop

WikiGame

the environment · not the agent

3 Track the task

current_page · page_history
goal_page

check_win() is WikiGame's method: it answers “on the goal?”. The loop is what calls it.

Q&A + polls on Slido →

THE SPLIT

Seven jobs: six in the agent, one in the environment

Tools are your Python too, so they belong to the scaffold. The model itself is none of these.

1 Call the LLM | 2 Tools | 3 Track the task | 4 Memory | 5 Read the reply | 6 Decide what's next | 7 Task info

THE AGENT'S SCAFFOLD · all your Python

Tools

functions the model asks for

2 Tools

GetContentTool(game)
MovePageTool(game)

They read and change the board.

WikiAgent

the loop

WikiGame

the environment · not the agent

3 Track the task

current_page · page_history
goal_page

check_win() is WikiGame's method: it answers “on the goal?”. The loop is what calls it.

Q&A + polls on Slido →

THE SPLIT

Seven jobs: six in the agent, one in the environment

Tools are your Python too, so they belong to the scaffold. The model itself is none of these.

1 Call the LLM | 2 Tools | 3 Track the task | 4 Memory | 5 Read the reply | 6 Decide what's next | 7 Task info

THE AGENT'S SCAFFOLD · all your Python

Tools

functions the model asks for

2 Tools

GetContentTool(game)
MovePageTool(game)

They read and change the board.

WikiAgent

the loop

1	Call the LLM	get_model().generate()
4	Memory	state.messages
5	Read the reply	_handle_tool_calls()
6	Decide what's next	execute()
7	Task info	the three prompts

WikiGame

the environment · not the agent

3 Track the task

current_page · page_history
goal_page

check_win() is WikiGame's method: it answers “on the goal?”. The loop is what calls it.

Q&A + polls on Slido →

ENVIRONMENT OR SCAFFOLD?

The board knows. The loop decides.

WikiGame holds the facts and rules. Your scaffold asks it questions, then decides what to do.

ENVIRONMENT · WikiGame

Holds the facts and the rules. Never acts on its own.

current_page, page_history
where we are, and how we got here

is_permitted_link(page)
is this a legal click? (a link in the main text: in two slides)

check_win()
answers: is current_page the goal page?

get_page(), get_permitted_links()
fetch pages and links from Wikipedia

SCAFFOLD · WikiAgent + tools

Your Python. Asks the board, then decides.

writes the prompts
from game.current_page and the goal

calls the model
sends the chat and the tool descriptions

runs the tools
your Python; they read and move the board

calls game.check_win()
and decides: stop, or go round again

Q&A + polls on Slido →

ENVIRONMENT OR SCAFFOLD?

The board knows. The loop decides.

WikiGame holds the facts and rules. Your scaffold asks it questions, then decides what to do.

ENVIRONMENT · WikiGame

Holds the facts and the rules. Never acts on its own.

current_page, page_history
where we are, and how we got here

is_permitted_link(page)
is this a legal click? (a link in the main text: in two slides)

check_win()
answers: is current_page the goal page?

get_page(), get_permitted_links()
fetch pages and links from Wikipedia

SCAFFOLD · WikiAgent + tools

Your Python. Asks the board, then decides.

writes the prompts
from game.current_page and the goal

calls the model
sends the chat and the tool descriptions

runs the tools
your Python; they read and move the board

calls game.check_win()
and decides: stop, or go round again

Q&A + polls on Slido →

The board knows. The loop decides.

WikiGame holds the facts and rules. Your scaffold asks it questions, then decides what to do.

ENVIRONMENT · WikiGame

Holds the facts and the rules. Never acts on its own.

current_page, page_history
where we are, and how we got here

is_permitted_link(page)
is this a legal click? (a link in the main text: in two slides)

check_win()
answers: is current_page the goal page?

get_page(), get_permitted_links()
fetch pages and links from Wikipedia

asks →

← answers

SCAFFOLD · WikiAgent + tools

Your Python. Asks the board, then decides.

writes the prompts
from game.current_page and the goal

calls the model
sends the chat and the tool descriptions

runs the tools
your Python; they read and move the board

calls game.check_win()
and decides: stop, or go round again

Phone analogy: the board is the room; your scaffold is the assistant in it; the model is the expert on the call.

Q&A + polls on Slido →

Why does the agent need a game board?

Problem first, then the fix: one object that everything shares.

Problem

The model only writes text. Something has to know:

the fix comes next…

Q&A + polls on Slido →

Why does the agent need a game board?

Problem first, then the fix: one object that everything shares.

Problem

The model only writes text. Something has to know:

- **which page are we on?**

the fix comes next…

Q&A + polls on Slido →

Why does the agent need a game board?

Problem first, then the fix: one object that everything shares.

Problem

The model only writes text. Something has to know:

- **which page are we on?**
- **is “India” a legal click here? (legal = in the main text, soon)**

the fix comes next…

Q&A + polls on Slido →

Why does the agent need a game board?

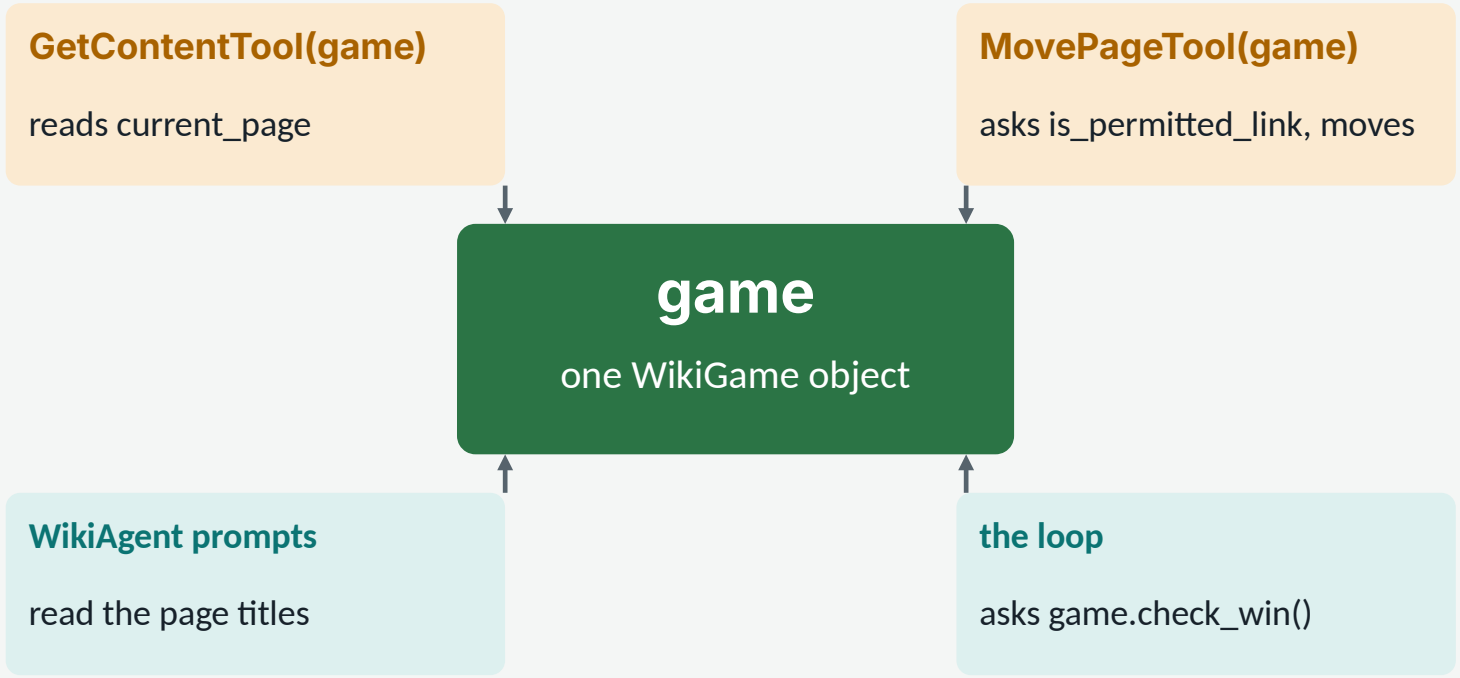
Problem first, then the fix: one object that everything shares.

Problem

The model only writes text. Something has to know:

- **which page are we on?**
- **is “India” a legal click here? (legal = in the main text, soon)**
- **have we reached the goal?**

And the chat can't hold it: we wipe it after every move.




```
game = WikiGame("Mango", "Chandrayaan-3")  # the board
tool_list = [GetContentTool(game), MovePageTool(game)]  # same game in both
```


Q&A + polls on Slido →

THE ENVIRONMENT

WikiGame: the game board, as a Python class

Given in the notebook, not an exercise. One WikiGame per game; both tools and the loop share it.

It remembers

start: Mango

goal: Chandrayaan-3

current_page

page_history

It can, when asked

get_page(title): fetch a page

get_page_summary(): first ~500 characters

get_permitted_links(): the legal links

is_permitted_link(link)

check_win(): on the goal?

Who asks it

GetContentTool reads current_page and its links

MovePageTool asks is_permitted_link, then moves it

The prompts read the titles and the goal summary

The loop asks check_win()

The model never touches it.

Q&A + polls on Slido →

THE ENVIRONMENT

WikiGame: the game board, as a Python class

Given in the notebook, not an exercise. One WikiGame per game; both tools and the loop share it.

It remembers

start: Mango

goal: Chandrayaan-3

current_page

page_history

It can, when asked

get_page(title): fetch a page

get_page_summary(): first ~500 characters

get_permitted_links(): the legal links

is_permitted_link(link)

check_win(): on the goal?

Who asks it

GetContentTool reads current_page and its links

MovePageTool asks is_permitted_link, then moves it

The prompts read the titles and the goal summary

The loop asks check_win()

The model never touches it.

Q&A + polls on Slido →

THE ENVIRONMENT

WikiGame: the game board, as a Python class

Given in the notebook, not an exercise. One WikiGame per game; both tools and the loop share it.

It remembers

start: Mango

goal: Chandrayaan-3

current_page

page_history

It can, when asked

get_page(title): fetch a page

get_page_summary(): first ~500 characters

get_permitted_links(): the legal links

is_permitted_link(link)

check_win(): on the goal?

Who asks it

GetContentTool reads current_page and its links

MovePageTool asks is_permitted_link, then moves it

The prompts read the titles and the goal summary

The loop asks check_win()

The model never touches it.

Q&A + polls on Slido →

THE ENVIRONMENT

WikiGame: the game board, as a Python class

Given in the notebook, not an exercise. One WikiGame per game; both tools and the loop share it.

It remembers

start: Mango

goal: Chandrayaan-3

current_page

page_history

It can, when asked

get_page(title): fetch a page

get_page_summary(): first ~500 characters

get_permitted_links(): the legal links

is_permitted_link(link)

check_win(): on the goal?

Who asks it

GetContentTool reads current_page and its links

MovePageTool asks is_permitted_link, then moves it

The prompts read the titles and the goal summary

The loop asks check_win()

The model never touches it.

Board vs chat: WikiGame is where the agent really is. The chat is only what it has read and said.

Q&A + polls on Slido →

WHICH LINKS COUNT

Only links in the main text count

Wikipedia lists every link on a page, sidebars included. The notebook filters them.

Links on the Mango page (illustrative)

India	ISBN (identifier)
Asia	Wayback Machine
Indian cuisine	Wikidata
Lassi	List of mango cultivars

Green: in the main text, allowed. Grey: sidebar and references.

The trick

Keep a link only if its title appears in the page's text.

Where it misses

Pages often use a shorter name: “ISRO” never matches “Indian Space Research Organisation”. Pick goals mentioned by their exact title.

Q&A + polls on Slido →

WHY GET_PAGE EXISTS

The model types page titles. Wikipedia wants exact ones.

The model writes the title in MovePageTool(page="..."). A near-miss makes wikipedia.page() raise an error.

“Animalss”

a typo

→ **PageError**

no page has exactly this title

“Python”

many pages share the name

→ **DisambiguationError**

the language? the snake? Wikipedia won't pick

“Bombay”

an old or other name

→ **a redirect**

Wikipedia's own “this name means that page” link

SO THE NOTEBOOK GIVES YOU

WikiGame.get_page()

1. try the exact title, following redirects
2. ambiguous? take the first option
3. not found? let Wikipedia guess (auto_suggest)

You don't write this. But you'll see these errors in your logs, so know where they come from.

Q&A + polls on Slido →

When the model gets a page name slightly wrong

“Bombay”, “huMan”, “gogle”: names that aren't an exact title. Two ways to fix them.

Follow redirects

Wikipedia's own “this name means that page” links. Safe.

"Bombay" → Mumbai

"huMan" → Human

Search and guess

Takes Wikipedia's top search result. Can be wrong, with no error.

"gogle" → Google

"Human" → Man ×

WikiGame's get_page (given): follow redirects first; search only if that fails. You don't write this, but you'll see these errors in logs.

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 3.1 the game and the board

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

The board is the environment

WikiGame holds where we are, the rules, and whether we've won. It never acts.

WikiGame

2

The scaffold asks it

Tools and the loop are your Python. They read the board, move it, and call check_win().

loop + tools = scaffold

3

Legal = main text

Only links in the page's main text count. get_page() fixes near-miss titles.

get_permitted_links()

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

Where does check_win() live, and who calls it?

A. In the model: it decides it has won

B. In WikiGame, and the loop calls it

C. In the tools: MovePageTool calls it

D. In Inspect: eval() calls it each step

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

3.2 Tools in Inspect

Questions on the last section? Ask in the room or on Slido.

THIS PART

3.1 The game and the board ✓

3.2 Tools in Inspect

3.3 From solver to agent: what changes

3.4 The loop, step by step

ASK NOW

Anything from the last section still unclear? This is the moment.

Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

Anatomy of a tool: MovePageTool

The body is pseudocode: writing it is your exercise. Four parts, four jobs.

```
@tool
def MovePageTool(game: WikiGame) → Tool:
    async def execute(page: str) → str:
        """
        Move to a new wikipedia page by clicking on a
        link in the current page content.

        Args:
            page: The title of the page you want to move to.
        """
        # pseudocode: this part is your exercise
        # if page is a permitted link on the current page:
        #     move the board there, add it to the history
        #     return a success message
        # otherwise: return a failure message
    return execute
```

1 OF 4 · THE NAME

Outer function name = the tool's name

The model calls it as “MovePageTool”.

game is fixed when you build the tool: MovePageTool(game). The model never sees it.

Q&A + polls on Slido →

Anatomy of a tool: MovePageTool

The body is pseudocode: writing it is your exercise. Four parts, four jobs.

```
@tool
def MovePageTool(game: WikiGame) → Tool:
    async def execute(page: str) → str:
        """
        Move to a new wikipedia page by clicking on a
        link in the current page content.

        Args:
            page: The title of the page you want to move to.
        """
        # pseudocode: this part is your exercise
        # if page is a permitted link on the current page:
        #     move the board there, add it to the history
        #     return a success message
        # otherwise: return a failure message
    return execute
```

2 OF 4 · THE ARGUMENTS

execute's signature = the arguments

page: str means the model must send {"page": "..."}.

Type hints are required: Inspect reads them.

Q&A + polls on Slido →

Anatomy of a tool: MovePageTool

The body is pseudocode: writing it is your exercise. Four parts, four jobs.

```
@tool
def MovePageTool(game: WikiGame) → Tool:
    async def execute(page: str) → str:
        """
        Move to a new wikipedia page by clicking on a
        link in the current page content.

        Args:
            page: The title of the page you want to move to.
        """
        # pseudocode: this part is your exercise
        # if page is a permitted link on the current page:
        #     move the board there, add it to the history
        #     return a success message
        # otherwise: return a failure message
    return execute
```

3 OF 4 · THE MANUAL

The docstring = the description

It's the model's only manual for the tool.

Every argument needs an Args: line, or Inspect raises an error.

Q&A + polls on Slido →

Anatomy of a tool: MovePageTool

The body is pseudocode: writing it is your exercise. Four parts, four jobs.

```
@tool
def MovePageTool(game: WikiGame) → Tool:
    async def execute(page: str) → str:
        """
        Move to a new wikipedia page by clicking on a
        link in the current page content.

        Args:
            page: The title of the page you want to move to.
        """
        # pseudocode: this part is your exercise
        # if page is a permitted link on the current page:
        #     move the board there, add it to the history
        #     return a success message
        # otherwise: return a failure message
    return execute
```

4 OF 4 · THE RESULT

The return value = the tool message

Your code (the exercise) changes the board, then returns a short string.

That string, e.g. “Move successful”, goes back into the chat as a tool message.

Q&A + polls on Slido →

WHAT THE MODEL SEES

Your docstring becomes the tool description

Inspect builds this JSON, not your scaffold. Your scaffold just passes tools=[...] to generate().

YOU WRITE THIS · the tool

```
@tool
def MovePageTool(game):
    async def execute(page: str) → str:
        """
        Move to a new wikipedia page by
        clicking on a link in the current
        page content.
        Args:
            page: The title of the page you
              want to move to.
        """
```

INSPECT BUILDS THIS · sent on every generate(tools=...) call

```
{"type": "function",
 "function": {
  "name": "MovePageTool",
  "description": "Move to a new wikipedia
    page by clicking on a link...",
  "parameters": {"properties": {
    "page": {"type": "string",
      "description": "The title of the page
      you want to move to."}},
  "required": ["page"]}}}
```

name ← outer function name

description ← docstring, 1st paragraph

type ← the type hint: str

page + its description ← the Args: line

No Args: line for an argument? Inspect raises **ValueError: Description not provided for parameter**

Q&A + polls on Slido →

LIVE DEMO · INTERACTIVE PAGE

Tool Schema Lab

Edit a tool's code and watch the JSON the model receives change, and what Inspect rejects.

OPEN

tool_schema_lab.html

Click the name to open. Also linked on the last slide. Any browser, nothing to install.

TRY THIS

1. Load “Broken: no Args line”: what error would Inspect raise?
2. Rename the outer function. What does the model now call?
3. Delete half the docstring: what is the model left with?

Q&A + polls on Slido →

THE TOOLS

The agent gets exactly two tools

Everything it does in the game goes through one of these. You write both.

GetContentTool()

Returns the current page's text, with every allowed link in tags:

“…grown in <link>India</link> for thousands of years…”

MovePageTool(page)

Allowed link? Move, and reply “Move successful”.

Not allowed? “Move failed…”, pointing to the <link> tags.

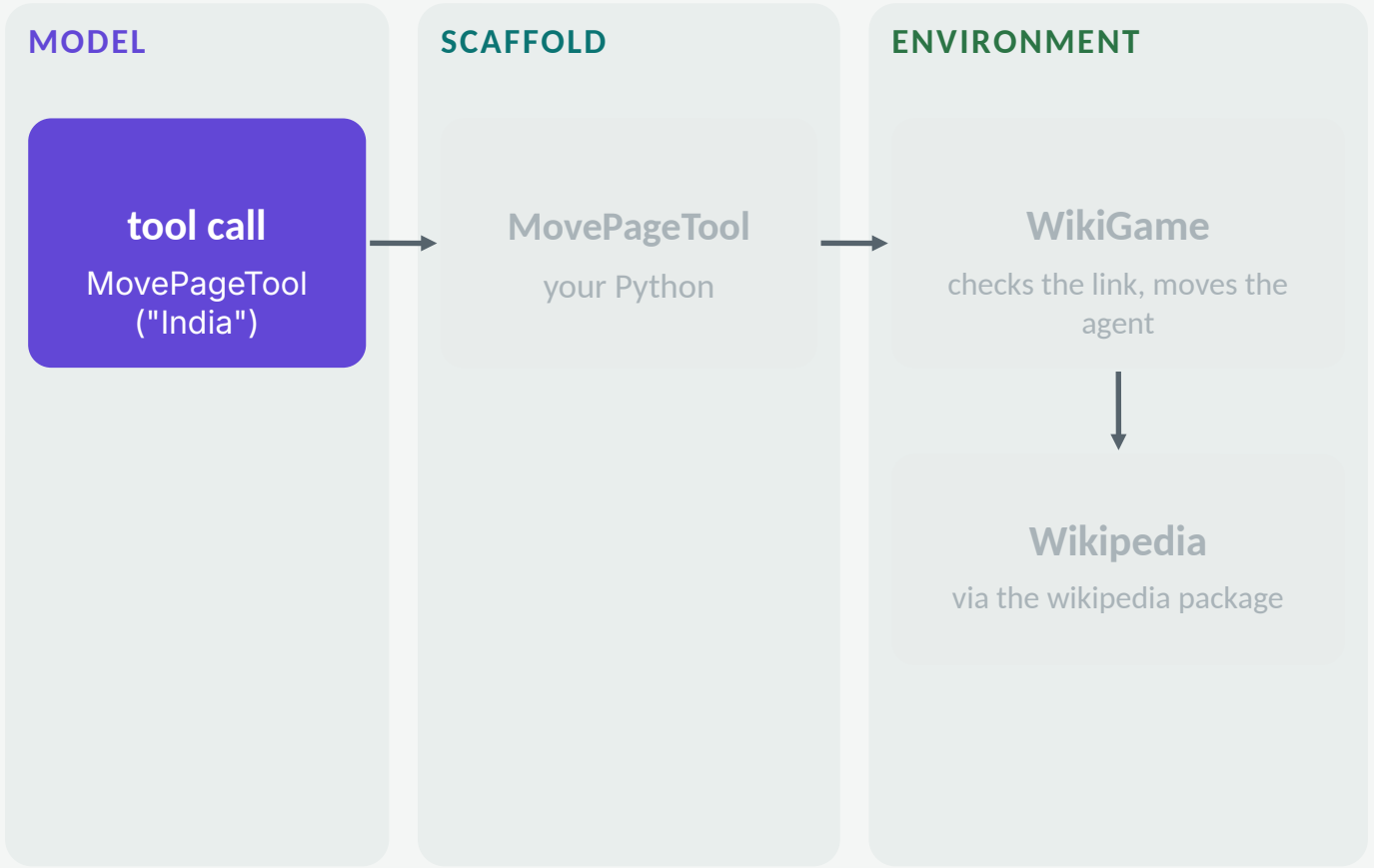
Why not just give it a list of links?

It would do a little better, but reading the page and spotting the right link is part of the game.

Q&A + polls on Slido →

One move, end to end: who does what

Mango → India, through the model, the scaffold and the environment.



STEP 1 OF 4

1 · The model asks to move

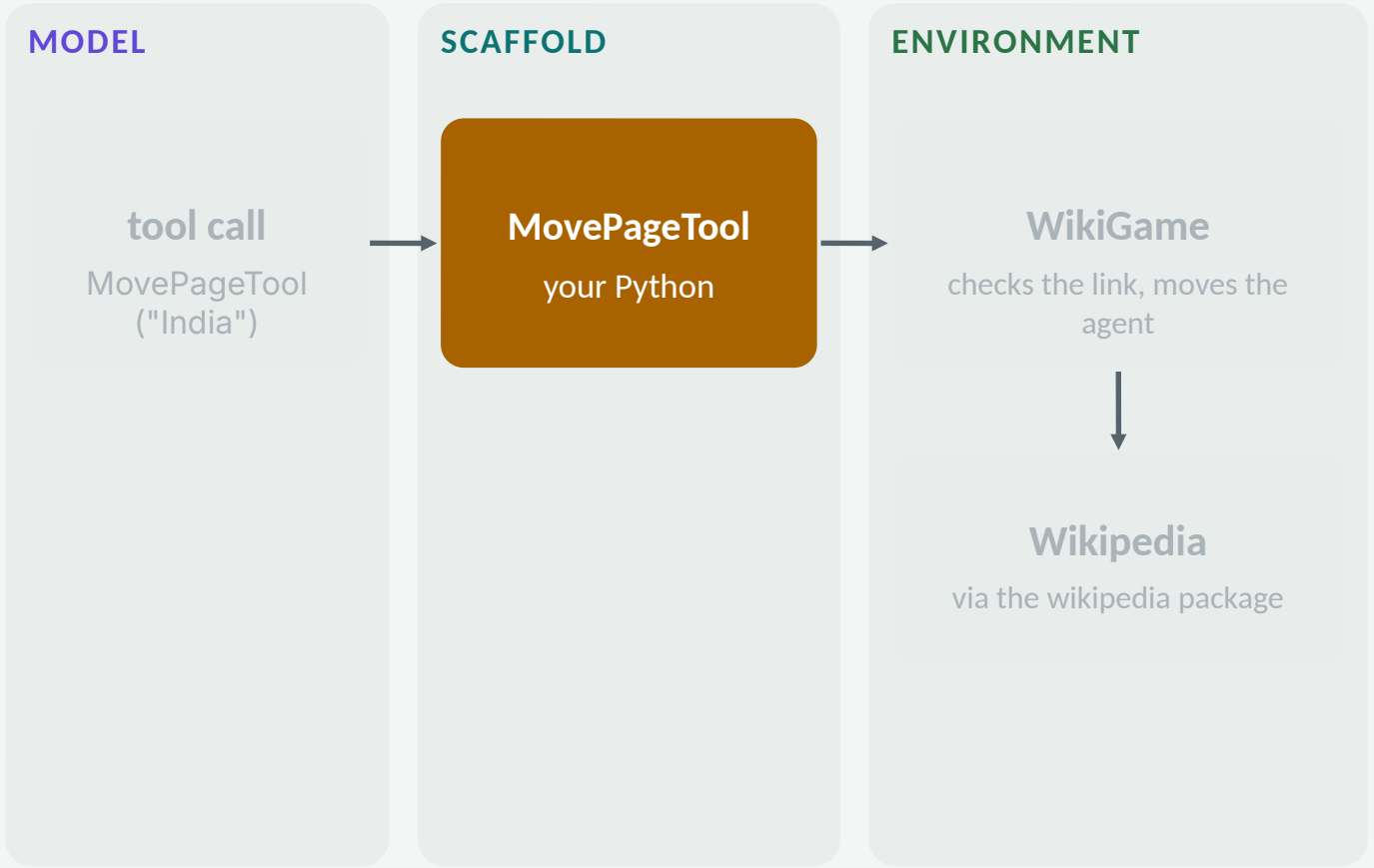
It writes a tool call: MovePageTool with page “India”. Just text.

It never talks to Wikipedia itself.

Q&A + polls on Slido →

One move, end to end: who does what

Mango → India, through the model, the scaffold and the environment.



STEP 2 OF 4

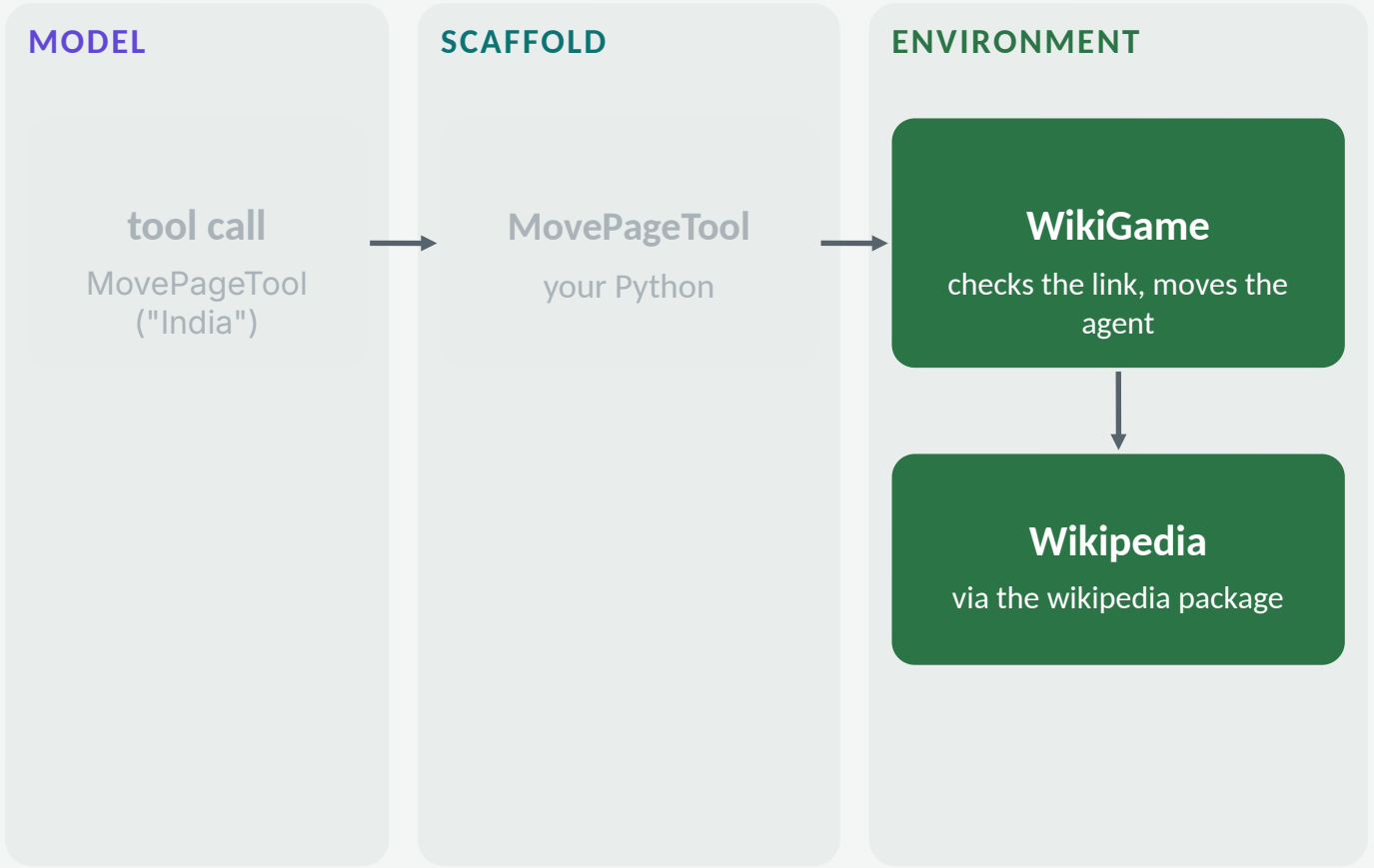
2 · Your code runs the tool

MovePageTool is your Python, so it's part of the **scaffold**.

Q&A + polls on Slido →

One move, end to end: who does what

Mango → India, through the model, the scaffold and the environment.



STEP 3 OF 4

3 · The environment does the move

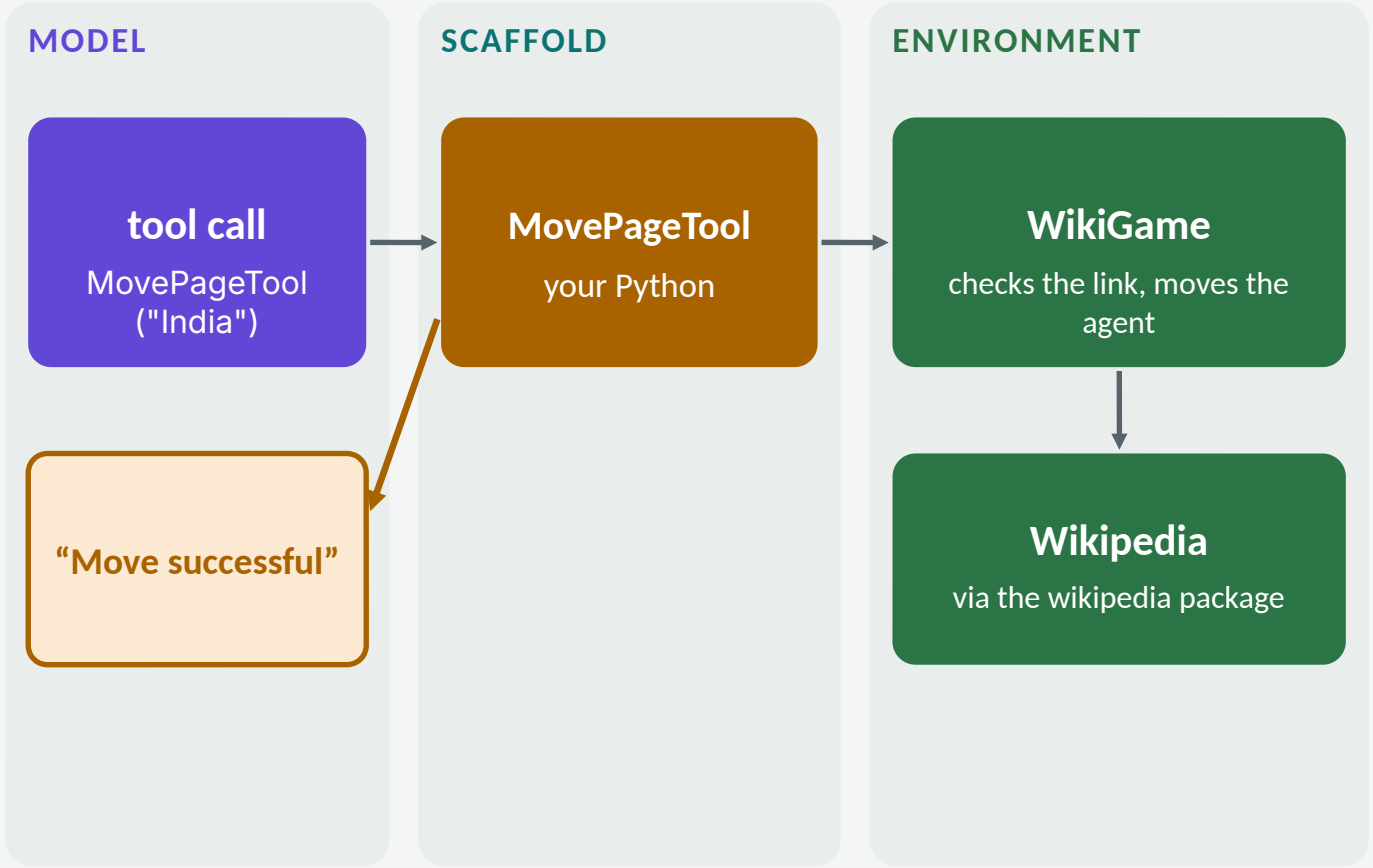
WikiGame checks India is a link on the Mango page, fetches the page (using the wikipedia Python library), and now says the agent is on India.

That call to Wikipedia happens inside our code. It is not a tool call.

Q&A + polls on Slido →

One move, end to end: who does what

Mango → India, through the model, the scaffold and the environment.



ALL TOGETHER

4 · A short result goes back

The tool replies "Move successful", which is added to the conversation.

The model only asked. Your code did the work.

Q&A + polls on Slido →

AFTER THE TOOL RUNS

How the result gets back to the model

The model can't see the board. It only learns what happened from a tool message your scaffold adds to the chat.

state.messages · the chat

system	You are a wikipedia-racing AI…
user	You are currently on page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → MovePageTool(page="India") id: call_1
tool	“Move successful” answers: call_1

Failed move? Same thing: “Move failed, link not permitted…” is the tool message, so the model sees its mistake.

1

The tool returns a string

MovePageTool's execute returns “Move successful”.

2

execute_tools() wraps it

as a tool message, linked to the call that asked for it (same id).

3

Your loop adds it to the chat

state.messages.extend(…). Now the next call will carry it.

Q&A + polls on Slido →

ASKING FOR THE NEXT STEP

How the scaffold asks the model what to do next

The model never asks for a turn. Your loop adds a prompt and calls generate() again, with the whole chat.

After a read or a failed move

system	You are a wikipedia-racing AI…
user	You are currently on page: Mango…
…	
tool	“Move failed, link not permitted…”
user	What will you do next?

The tool message stays. The model reads it, then answers.

After a successful move

system	You are a wikipedia-racing AI…
user	You are currently on page: India…
user	What will you do next?

Prompts rebuilt from the board, old chat wiped. It learns the move worked from the new page name.

Then: get_model().generate(input=state.messages, tools=tools) → the next reply: another tool call, or text.

Q&A + polls on Slido →

ASKING FOR THE NEXT STEP

How the scaffold asks the model what to do next

The model never asks for a turn. Your loop adds a prompt and calls generate() again, with the whole chat.

After a read or a failed move

system	You are a wikipedia-racing AI…
user	You are currently on page: Mango…

…

tool	“Move failed, link not permitted…”
user	What will you do next?

The tool message stays. The model reads it, then answers.

After a successful move

system	You are a wikipedia-racing AI…
user	You are currently on page: India…
user	What will you do next?

Prompts rebuilt from the board, old chat wiped. It learns the move worked from the new page name.

Then: get_model().generate(input=state.messages, tools=tools) → the next reply: another tool call, or text.

Q&A + polls on Slido →

ASKING FOR THE NEXT STEP

How the scaffold asks the model what to do next

The model never asks for a turn. Your loop adds a prompt and calls generate() again, with the whole chat.

After a read or a failed move

system	You are a wikipedia-racing AI…
user	You are currently on page: Mango…
…	
tool	“Move failed, link not permitted…”
user	What will you do next?

The tool message stays. The model reads it, then answers.

After a successful move

system	You are a wikipedia-racing AI…
user	You are currently on page: India…
user	What will you do next?

Prompts rebuilt from the board, old chat wiped. It learns the move worked from the new page name.

Then: get_model().generate(input=state.messages, tools=tools) → the next reply: another tool call, or text.

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 3.2 tools

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

A tool is name + args + docstring

That's all the model sees: Inspect turns them into JSON.

`@tool`

2

The model only asks

It writes a tool call. execute_tools() runs it in your Python, on the board.

`execute_tools()`

3

Results come back as messages

Your loop adds the tool message, then asks “What will you do next?”.

`tool message`

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

MovePageTool returns “Move failed…”. How does the model find out?

- **A** It watches the board change
- **B** Wikipedia sends it an error
- **C** Inspect retries the move for it
- **D** The tool message is in its next chat

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

3.3 From solver to agent: what changes

Questions on the last section? Ask in the room or on Slido.

THIS PART

3.1 The game and the board ✓

3.2 Tools in Inspect ✓

3.3 From solver to agent: what changes

3.4 The loop, step by step

ASK NOW

Anything from the last section still unclear? This is the moment.

Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

Last week's solver → this week's agent, as a diff

Pseudocode, with the Inspect calls as comments. Red lines go, green lines come in. Four changes.

```
- @solver
+ @agent
- def my_solver():
+ def WikiAgent(tools, game):
-     async def solve(state: TaskState, generate):
+     async def execute(state: AgentState):
-         call the model once      # generate(state)
+         repeat until game.check_win():
+             ask the model, with the tools
+                 # get_model().generate(..., tools=tools)
+             add its reply to state.messages  # you, not Inspect
+             if it asked for a tool: run it
          return state
-     return solve
+     return execute
```

CHANGE 1 OF 4

@agent instead of @solver

Same shape: an outer function that returns an inner one.

The outer one holds the setup: here, the tools and the game.

Q&A + polls on Slido →

Last week's solver → this week's agent, as a diff

Pseudocode, with the Inspect calls as comments. Red lines go, green lines come in. Four changes.

```
- @solver
+ @agent
- def my_solver():
+ def WikiAgent(tools, game):
-     async def solve(state: TaskState, generate):
+     async def execute(state: AgentState):
-         call the model once      # generate(state)
+         repeat until game.check_win():
+             ask the model, with the tools
+                 # get_model().generate(..., tools=tools)
+             add its reply to state.messages  # you, not Inspect
+             if it asked for a tool: run it
          return state
-     return solve
+     return execute
```

CHANGE 2 OF 4

AgentState, and no generate

The inner function gets an **AgentState**: just messages and output.

There's no generate argument any more.

Q&A + polls on Slido →

SOLVER → AGENT

Last week's solver → this week's agent, as a diff

Pseudocode, with the Inspect calls as comments. Red lines go, green lines come in. Four changes.

```
- @solver
+ @agent
- def my_solver():
+ def WikiAgent(tools, game):
-     async def solve(state: TaskState, generate):
+     async def execute(state: AgentState):
-         call the model once     # generate(state)
+         repeat until game.check_win():
+             ask the model, with the tools
+                 # get_model().generate(..., tools=tools)
+             add its reply to state.messages  # you, not Inspect
+             if it asked for a tool: run it
          return state
-     return solve
+     return execute
```

CHANGE 3 OF 4

You call the model yourself

The agent calls the model itself, and hands it the tools it may ask for.

Then you append the reply yourself. Inspect won't.

Q&A + polls on Slido →

Last week's solver → this week's agent, as a diff

Pseudocode, with the Inspect calls as comments. Red lines go, green lines come in. Four changes.

```
- @solver
+ @agent
- def my_solver():
+ def WikiAgent(tools, game):
-     async def solve(state: TaskState, generate):
+     async def execute(state: AgentState):
-         call the model once      # generate(state)
+         repeat until game.check_win():
+             ask the model, with the tools
+                 # get_model().generate(..., tools=tools)
+             add its reply to state.messages  # you, not Inspect
+             if it asked for a tool: run it
          return state
-     return solve
+     return execute
```

CHANGE 4 OF 4

A loop, not one step

If the reply is a tool call, run it. Then check the board.

Repeat until check_win(), or the Task's message_limit.

Q&A + polls on Slido →

WHAT CHANGES

Last week vs this week, piece by piece

Same Inspect. What each piece becomes for an agent.

	LAST WEEK · POWER-SEEKING MCQS	THIS WEEK · WIKIAGENT
Dataset	300 Samples, one per question	**one empty Sample: the task lives in WikiGame**
Solver	a fixed chain of solvers	**as_solver(WikiAgent(...)): one solver, a loop inside**
State		
Model call		
Tools		
Stopping		
Scorer		

Q&A + polls on Slido →

WHAT CHANGES

Last week vs this week, piece by piece

Same Inspect. What each piece becomes for an agent.

	LAST WEEK · POWER-SEEKING MCQS	THIS WEEK · WIKIAGENT
Dataset	300 Samples, one per question	**one empty Sample: the task lives in WikiGame**
Solver	a fixed chain of solvers	**as_solver(WikiAgent(...)): one solver, a loop inside**
State	TaskState: messages, choices, target…	**AgentState: just messages and output**
Model call	the generate() solver	**your loop calls the model itself, with the tools**
Tools		
Stopping		
Scorer		

Q&A + polls on Slido →

WHAT CHANGES

Last week vs this week, piece by piece

Same Inspect. What each piece becomes for an agent.

	LAST WEEK · POWER-SEEKING MCQS	THIS WEEK · WIKIAGENT
Dataset	300 Samples, one per question	**one empty Sample: the task lives in WikiGame**
Solver	a fixed chain of solvers	**as_solver(WikiAgent(...)): one solver, a loop inside**
State	TaskState: messages, choices, target…	**AgentState: just messages and output**
Model call	the generate() solver	**your loop calls the model itself, with the tools**
Tools	none	**@tool functions, run by execute_tools()**
Stopping	after the last solver	**check_win(), or message_limit**
Scorer		

Q&A + polls on Slido →

WHAT CHANGES

Last week vs this week, piece by piece

Same Inspect. What each piece becomes for an agent.

	LAST WEEK · POWER-SEEKING MCQS	THIS WEEK · WIKIAGENT
Dataset	300 Samples, one per question	**one empty Sample: the task lives in WikiGame**
Solver	a fixed chain of solvers	**as_solver(WikiAgent(...)): one solver, a loop inside**
State	TaskState: messages, choices, target…	**AgentState: just messages and output**
Model call	the generate() solver	**your loop calls the model itself, with the tools**
Tools	none	**@tool functions, run by execute_tools()**
Stopping	after the last solver	**check_win(), or message_limit**
Scorer	reads the reply: C or I	**none here: the loop checks the win itself**

Q&A + polls on Slido →

WHAT INSPECT NEEDS

To run anything, Inspect needs a Task

Same as last week: a Task bundles a dataset and a solver (and usually a scorer). Here is what we give it, and why.

dataset

a list of Samples

dataset=[Sample(input="", target="")]

One Sample, and it's empty. The real task (start, goal, the pages) lives in WikiGame, and the prompts come from your scaffold.

solver

what runs on each Sample

solver=as_solver(WikiAgent(...))

Our whole agent, wrapped so Inspect can run it like last week's solvers.

scorer + limit

how to mark it; when to stop

message_limit=80 # no scorer

No scorer here: the loop stops itself at check_win(). The limit stops a runaway loop.

Q&A + polls on Slido →

WHAT INSPECT NEEDS

To run anything, Inspect needs a Task

Same as last week: a Task bundles a dataset and a solver (and usually a scorer). Here is what we give it, and why.

dataset

a list of Samples

`dataset=[Sample(input="", target="")]`

One Sample, and it's empty. The real task (start, goal, the pages) lives in WikiGame, and the prompts come from your scaffold.

solver

what runs on each Sample

`solver=as_solver(WikiAgent(...))`

Our whole agent, wrapped so Inspect can run it like last week's solvers.

scorer + limit

how to mark it; when to stop

`message_limit=80  # no scorer`

No scorer here: the loop stops itself at check_win(). The limit stops a runaway loop.

Q&A + polls on Slido →

WHAT INSPECT NEEDS

To run anything, Inspect needs a Task

Same as last week: a Task bundles a dataset and a solver (and usually a scorer). Here is what we give it, and why.

dataset

a list of Samples

```
dataset=[Sample(input="", target="")]
```

One Sample, and it's empty. The real task (start, goal, the pages) lives in WikiGame, and the prompts come from your scaffold.

solver

what runs on each Sample

```
solver=as_solver(WikiAgent(...))
```

Our whole agent, wrapped so Inspect can run it like last week's solvers.

scorer + limit

how to mark it; when to stop

```
message_limit=80  # no scorer
```

No scorer here: the loop stops itself at check_win(). The limit stops a runaway loop.

Q&A + polls on Slido →

WHAT INSPECT NEEDS

To run anything, Inspect needs a Task

Same as last week: a Task bundles a dataset and a solver (and usually a scorer). Here is what we give it, and why.

dataset

a list of Samples

dataset=[Sample(input="", target="")]

One Sample, and it's empty. The real task (start, goal, the pages) lives in WikiGame, and the prompts come from your scaffold.

solver

what runs on each Sample

solver=as_solver(WikiAgent(...))

Our whole agent, wrapped so Inspect can run it like last week's solvers.

scorer + limit

how to mark it; when to stop

message_limit=80 # no scorer

No scorer here: the loop stops itself at check_win(). The limit stops a runaway loop.

Q&A + polls on Slido →

FROM EMPTY SAMPLE TO FIRST PROMPT

Where do the first messages come from?

The empty Sample becomes one empty user message. Your scaffold writes everything the model actually reads.

Sample

`Sample(input="")`

what we gave Inspect

→

TaskState

`messages = [user ""]`

Inspect makes it for each Sample. You never write it.

→

AgentState

`messages = [user ""]`

as_solver copies the messages in. This is all your agent sees.

→

your _start()

`+ system  + on-page`

your scaffold adds the first real prompts, written from the board

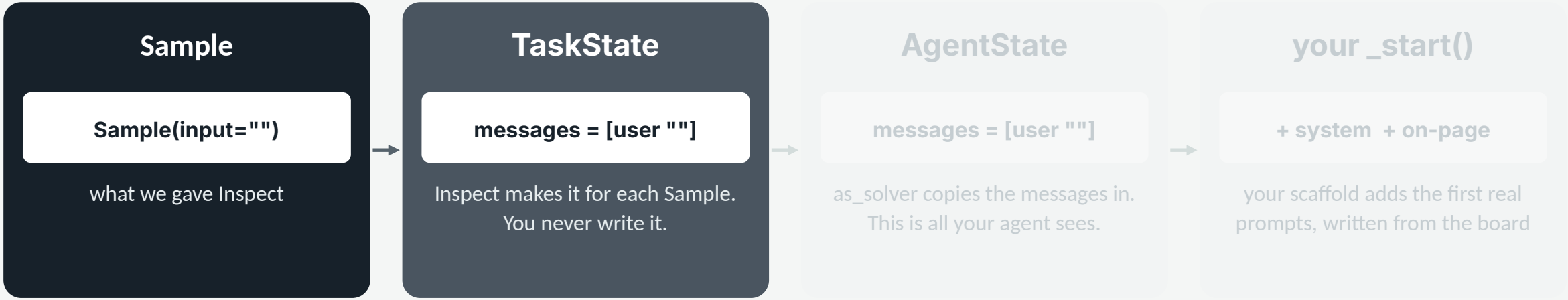
WHAT THE AGENT'S CHAT LOOKS LIKE NOW

Q&A + polls on Slido →

FROM EMPTY SAMPLE TO FIRST PROMPT

Where do the first messages come from?

The empty Sample becomes one empty user message. Your scaffold writes everything the model actually reads.



WHAT THE AGENT'S CHAT LOOKS LIKE NOW

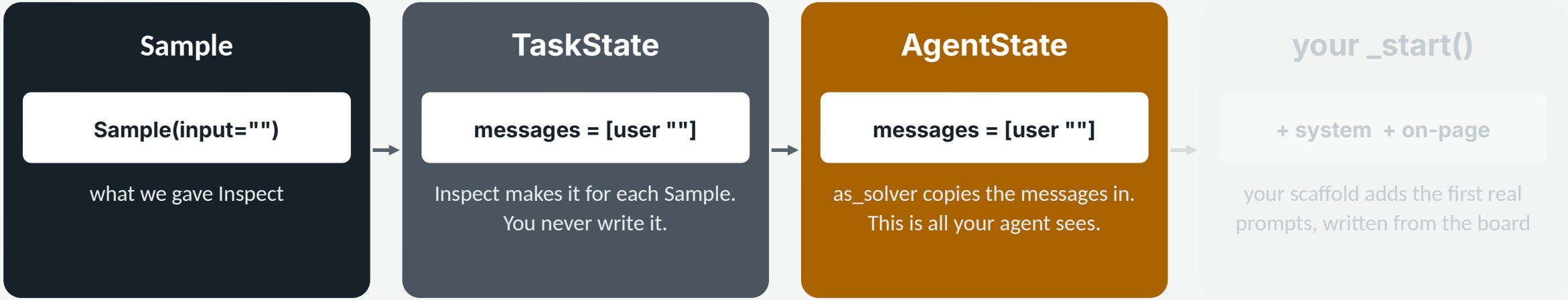
user "" ← the empty Sample input

Q&A + polls on Slido →

FROM EMPTY SAMPLE TO FIRST PROMPT

Where do the first messages come from?

The empty Sample becomes one empty user message. Your scaffold writes everything the model actually reads.



WHAT THE AGENT'S CHAT LOOKS LIKE NOW

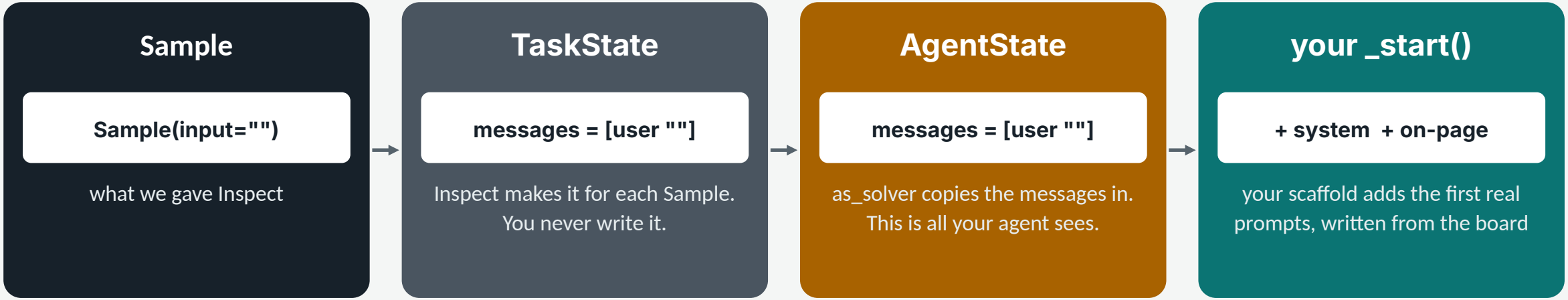
user "" ← the empty Sample input

Q&A + polls on Slido →

FROM EMPTY SAMPLE TO FIRST PROMPT

Where do the first messages come from?

The empty Sample becomes one empty user message. Your scaffold writes everything the model actually reads.



WHAT THE AGENT'S CHAT LOOKS LIKE NOW

user	"" ← the empty Sample input
system	You are a wikipedia-racing AI. Your aim is to reach the goal page…
user	You are currently on page: Mango. Your goal page is Chandrayaan-3.

Q&A + polls on Slido →

as_solver(): how your agent fits in eval()'s solver slot

Inspect's side, simplified: you never write this. TaskState is Inspect's record; your agent only ever sees the AgentState.

TaskState → AgentState → your loop → TaskState → log

```
def as_solver(agent):  # Inspect's code, simplified
    async def solve(task_state, generate):
        s = AgentState(messages=task_state.messages)
        s = await agent(s)
        task_state.messages = s.messages
        task_state.output = s.output
        return task_state
    return solve
```

STEP 1 OF 3 · COPY IN

Inspect made a TaskState; as_solver copies it in.

task_state.messages

user "" (the empty Sample input)

↓ copied into

s.messages

user ""

s.output = empty (no reply yet)

Q&A + polls on Slido →

as_solver(): how your agent fits in eval()'s solver slot

Inspect's side, simplified: you never write this. TaskState is Inspect's record; your agent only ever sees the AgentState.

TaskState → AgentState → **your loop** → TaskState → log

```
def as_solver(agent):  # Inspect's code, simplified
    async def solve(task_state, generate):
        s = AgentState(messages=task_state.messages)
        s = await agent(s)
        task_state.messages = s.messages
        task_state.output = s.output
        return task_state
    return solve
```

STEP 2 OF 3 · RUN YOUR AGENT

await = wait here until your whole loop returns.

s.messages while your loop runs

user	""
system	You are a wikipedia-racing AI...
user	You are currently on page: Mango...
user	What will you do next?
assistant	tool call → GetContentTool()
tool	The mango is... (whole page)

...wiped after every move, until check_win() or message_limit=80

Q&A + polls on Slido →

as_solver(): how your agent fits in eval()'s solver slot

Inspect's side, simplified: you never write this. TaskState is Inspect's record; your agent only ever sees the AgentState.

TaskState → AgentState → your loop → TaskState → log

```
def as_solver(agent):  # Inspect's code, simplified
    async def solve(task_state, generate):
        s = AgentState(messages=task_state.messages)
        s = await agent(s)
        task_state.messages = s.messages
        task_state.output = s.output
        return task_state
    return solve
```

STEP 3 OF 3 · COPY BACK

Messages and the last reply go back into the TaskState.

task_state after the run

system	You are a wikipedia-racing AI...
user	You are currently on page: Chandrayaan-3...

.output = the last reply

Because WikiAgent wipes the chat after each move, only the last page's chat comes back. **The log's Transcript still shows every model call.**

Q&A + polls on Slido →

PUTTING IT TOGETHER

The whole eval: board, tools, Task, eval()

This is the notebook's “run the task” cell.

```
game = WikiGame("Mango", "Chandrayaan-3")
tool_list = [GetContentTool(game), MovePageTool(game)]

@task
def wiki_task() → Task:
    return Task(dataset=[Sample(input="", target="")],
                message_limit=80)

eval(tasks=wiki_task(),
     solver=as_solver(WikiAgent(tools=tool_list, game=game)))
```

1 OF 3 · THE BOARD

One game, shared

Build the board first, then hand the same game to both tools.

Q&A + polls on Slido →

PUTTING IT TOGETHER

The whole eval: board, tools, Task, eval()

This is the notebook's “run the task” cell.

```
game = WikiGame("Mango", "Chandrayaan-3")
tool_list = [GetContentTool(game), MovePageTool(game)]

@task
def wiki_task() → Task:
   return Task(dataset=[Sample(input="", target="")],
               message_limit=80)

eval(tasks=wiki_task(),
     solver=as_solver(WikiAgent(tools=tool_list, game=game)))
```

2 OF 3 · THE TASK

One empty Sample, one limit

The input is empty: WikiAgent writes its own prompts from game.

message_limit=80 stops a runaway loop. Always set it.

Q&A + polls on Slido →

PUTTING IT TOGETHER

The whole eval: board, tools, Task, eval()

This is the notebook's “run the task” cell.

```
game = WikiGame("Mango", "Chandrayaan-3")
tool_list = [GetContentTool(game), MovePageTool(game)]

@task
def wiki_task() → Task:
    return Task(dataset=[Sample(input="", target="")],
                message_limit=80)

eval(tasks=wiki_task(),
     solver=as_solver(WikiAgent(tools=tool_list, game=game)))
```

3 OF 3 · EVAL()

as_solver puts the agent in the solver slot

One Sample → one run → one long conversation in the log.

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 3.3 solver → agent

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

Same Inspect, new pieces

@agent, AgentState, and your loop calls the model with the tools.

`@agent`

2

Inspect still needs a Task

One empty Sample, the agent as the solver, a message limit, no scorer.

`Task(dataset, solver)`

3

as_solver bridges the two

TaskState (Inspect's) → AgentState (yours) → run → copy back.

`as_solver(agent)`

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

Why is the Sample's input empty in wiki_task?

A The agent writes its own prompts from game

B Inspect needs every Sample input empty

C Tools only run when the input is empty

D The scorer needs it empty to spot a win

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

3.4 The loop, step by step

Questions on the last section? Ask in the room or on Slido.

THIS PART

- 3.1 The game and the board ✓
- 3.2 Tools in Inspect ✓
- 3.3 From solver to agent: what changes ✓
- **3.4 The loop, step by step**

ASK NOW

Anything from the last section still unclear? This is the moment.

Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

Inside WikiAgent: the pieces you write

The notebook's names. The small numbers are the seven jobs.

WikiAgent(tools, game)

PROMPTS 7

- **system_instruction**
- **on_page_instruction**
- **next_step_instruction**

Text for the model, written from the board (page, goal).

HELPERS

_start() 5

put system + on-page into the chat

_handle_tool_calls() 5

run tools; moved? refresh + reset

instruction_refresh() 7

rewrite prompts from the board

_reset_history() 4

wipe the chat, start again

THE LOOP 6

execute()

ask the model
↓
tool call? handle it
↓
won? stop

the model

only sees the chat and the tool descriptions

game

the board: read by the prompts, changed by the tools

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 1 OF 10

Start: tell it where it is

_start() writes two messages from the board. The model has no other way to know where it is.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.

game

current_page

Mango

page_history

Mango

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
   add "What will you do next?"
   reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 2 OF 10

Ask: what will you do next?

The whole chat, plus the two tool descriptions, goes to the API. The model writes one reply.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?

game

current_page

Mango

page_history

Mango

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 3 OF 10

The reply is just data: save it

ToolCall(function="GetContentTool", arguments={}). Nothing has run yet. Inspect won't save it for you.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → GetContentTool()

game

current_page

Mango

page_history

Mango

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 4 OF 10

Run the tool, add its result

Your Python reads the page off the board. It wasn't a move, so no wipe.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → GetContentTool()
tool	The mango is... <link>India</link>... (4k tok)

game

current_page

Mango

page_history

Mango

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
     if the reply asks for a tool:
         run it, add its result  # execute_tools
         if it was a successful move:
             rebuild prompts from game  # refresh
             wipe chat, restart with them  # reset
```

STEP 5 OF 10

Round 2: the model picks a link

Not won, so ask again. Now it can see the <link> tags.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → GetContentTool()
tool	The mango is... <link>India</link>... (4k tok)
user	What will you do next?
assistant	tool call → MovePageTool(page="India")

game

current_page

Mango

page_history

Mango

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
      run it, add its result  # execute_tools
       if it was a successful move:
           rebuild prompts from game  # refresh
           wipe chat, restart with them  # reset
```

STEP 6 OF 10

The tool moves the board

MovePageTool checks the link and moves the board to India. But the chat still says Mango.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → GetContentTool()
tool	The mango is... <link>India</link>... (4k tok)
user	What will you do next?
assistant	tool call → MovePageTool(page="India")
tool	Move successful

game

current_page

India

page_history

Mango

India

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
      if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 7 OF 10

Before the wipe

8 messages, about 4,100 tokens: mostly a page we've left. And the prompt names the wrong page.

state.messages · the chat — **≈ 4,100 tokens**

system	You are a wikipedia-racing AI...
user	On page: Mango. Goal: Chandrayaan-3.
user	What will you do next?
assistant	tool call → GetContentTool()
tool	The mango is... <link>India</link>... (4k tok)
user	What will you do next?
assistant	tool call → MovePageTool(page="India")
tool	Move successful

game

current_page: India

page_history: Mango, India

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 8 OF 10

Rebuild the prompts, wipe the chat

The chat forgets everything. The board doesn't: it still knows where we are and the path so far.

state.messages · the chat — ≈ 100 tokens

⟲ chat wiped by _reset_history()

system — You are a wikipedia-racing AI...

user — On page: India. Goal: Chandrayaan-3.

game

current_page: India

page_history: Mango, India

still knows ✓

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
    add "What will you do next?"
    reply = ask the model (chat + tools)
    add the reply to the chat
    if the reply asks for a tool:
        run it, add its result  # execute_tools
        if it was a successful move:
            rebuild prompts from game  # refresh
            wipe chat, restart with them  # reset
```

STEP 9 OF 10

Why wipe at all?

Pages are long, and every call re-sends the whole chat. Wiping keeps each call small. The price: the model also forgets its own plan.

state.messages · the chat

⟲ chat wiped by _reset_history()

system	You are a wikipedia-racing AI...
user	On page: India. Goal: Chandrayaan-3.

If we never wiped: the chat keeps every page read (≈ 4,000 tokens each) and re-sends it on every call
With the wipe: only the 2 fresh prompts, ≈ 100 tokens
Part 4 keeps the plan without the pages.

game

current_page

India

page_history

Mango

India

still knows ✓

Q&A + polls on Slido →

How the loop decides what's next, and moves on

WikiAgent's loop as pseudocode (turning it into code is your exercise). One run: read Mango, move to India.

```
add system + on-page messages  # _start()
repeat until game.check_win():
  add "What will you do next?"
  reply = ask the model (chat + tools)
  add the reply to the chat
  if the reply asks for a tool:
    run it, add its result  # execute_tools
    if it was a successful move:
      rebuild prompts from game  # refresh
      wipe chat, restart with them  # reset
```

STEP 10 OF 10

Check the board, go again

check_win(): India ≠ Chandrayaan-3. Round 3 starts from the fresh chat.

state.messages · the chat

system	You are a wikipedia-racing AI...
user	On page: India. Goal: Chandrayaan-3.
user	What will you do next?

game

current_page

India

page_history

Mango

India

Q&A + polls on Slido →

Reading an agent's log: the Transcript tab

One Sample = one long run. What a WikiAgent transcript looks like (sketch, simplified).

wiki_task · gpt-4o-mini · Sample 1 · Transcript

solver	as_solver(WikiAgent)
model	→ GetContentTool()
tool	GetContentTool → “The mango is…” (whole page)
model	→ MovePageTool(page="India")
tool	“Move successful”
model	→ MovePageTool(page="ISRO")
tool	“Move failed, link not permitted…”
model	→ GetContentTool()
…	… 40 more events …
limit	message_limit (80) reached

WHAT TO CHECK

Did it win, or hit the limit?

The last event tells you.

Count failed moves

“ISRO” isn't the link title: “Indian Space Research Organisation” is.

Same page twice?

It's going in circles: a history problem.

Read the chat after a move

Does the prompt name the new page?

Q&A + polls on Slido →

LIVE DEMO · INTERACTIVE PAGE

Log Viewer

A whole WikiAgent run, like Inspect's Transcript tab: the exact chat at every model call, and what changed.

OPEN

log_viewer.html

Click the name to open. Also linked on the last slide. Any browser, nothing to install.

TRY THIS

1. Find the first wipe: what disappears from the chat, and what doesn't?
2. Spot the failed move. Does the model see it on the next call?
3. Count the model calls. Which events didn't cost one?

Q&A + polls on Slido →

RECAP · PAUSE

Part 3 recap

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

Agent vs environment

WikiGame holds the game. The loop and the tools are the scaffold that asks it.

`scaffold ≠ environment`

2

Solver → agent

@agent, AgentState, you call the model and append its reply, and it loops.

`get_model().generate()`

3

One empty Sample

as_solver runs the whole loop as one solver. message_limit stops runaways.

`message_limit=80`

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

game.current_page is India, but the chat still says “You are currently on page: Mango”. What was skipped?

- **A** MovePageTool never updated the board
- **B** Wikipedia returned the wrong page
- **C** instruction_refresh() after the move
- **D** execute_tools() never ran the tool

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

PART 4 OF 5

Elicitation: getting the model's best

Before you say “it can't”, try hard to make it. Only the scaffold changes.

IN THIS PART

- Why elicit: every result is a floor
- Better prompts and ReAct
- Memory and a lookahead tool

AGENDA · ABOUT 75 MIN

1	Recap: from questions to agents ✓	8 min
2	What is an LLM agent? ✓	12 min
3	Agents in Inspect: the WikiGame ✓	25 min
4	**Elicitation**	**15 min**
5	Wrap-up and the notebook	5 min

Each part ends with a recap and a Slido poll.

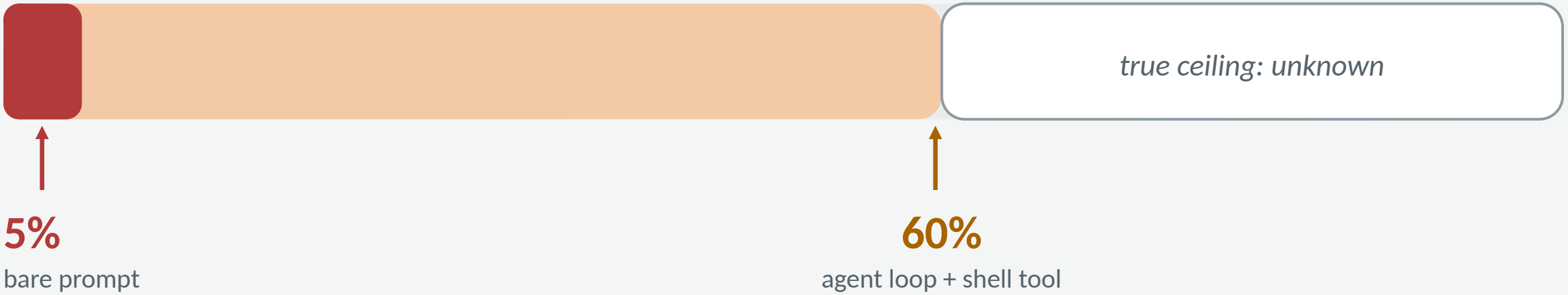


ELICITATION

Every eval result is a floor, not a ceiling

Elicitation = improving the scaffold to draw out the most the model can do, before judging it.

WARM-UP Q3, ANSWERED



What we can change:

- the prompts
- a prompt framework (ReAct)
- what's kept in history
- the tools

"Absence of evidence is not evidence of absence." Carl Sagan

Q&A + polls on Slido →

THE CORE IDEA

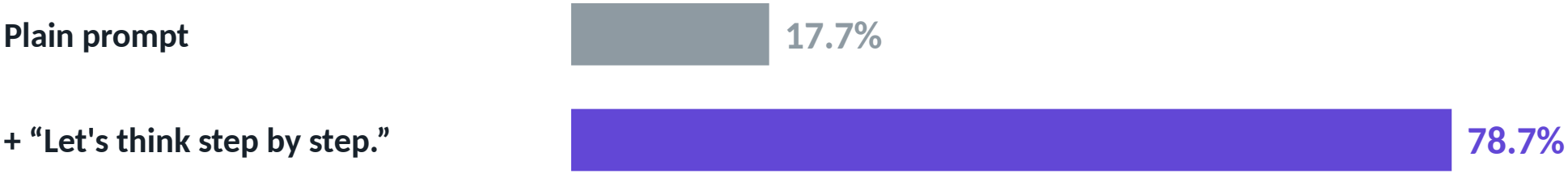
Not showing a skill ≠ not having it

Elicitation means deliberately trying to draw out the best a model can do, before judging it.

A failure is absence of evidence, not evidence of absence.

It tells you this model failed with this setup. Not that the model can't.

SAME MODEL, SAME MATHS PROBLEMS, ONE EXTRA SENTENCE (Kojima et al., 2022, MultiArith)



The honest claim is only: “we tried hard, with these methods, and couldn't make it happen.”

Q&A + polls on Slido →

WHERE IT FAILS

Our WikiAgent fails in three ways

It wins easy games (Elizabeth I → United States) and fails harder ones. Each failure points at a fix.

Bad prompt

It only knows the goal's title. No plan, no goal summary, no path so far.

FIX

better prompts · ReAct

Q&A + polls on Slido →

WHERE IT FAILS

Our WikiAgent fails in three ways

It wins easy games (Elizabeth I → United States) and fails harder ones. Each failure points at a fix.

Bad prompt

It only knows the goal's title. No plan, no goal summary, no path so far.

FIX

better prompts · ReAct

Bad history

The chat is wiped after every move, so it forgets its own plan and can walk in circles.

FIX

keep the history, drop old pages

Q&A + polls on Slido →

WHERE IT FAILS

Our WikiAgent fails in three ways

It wins easy games (Elizabeth I → United States) and fails harder ones. Each failure points at a fix.

Bad prompt

It only knows the goal's title. No plan, no goal summary, no path so far.

FIX

better prompts · ReAct

Bad history

The chat is wiped after every move, so it forgets its own plan and can walk in circles.

FIX

keep the history, drop old pages

Bad tools

A whole page at once: links deep in a long page get missed. No way to check a route first.

FIX

a lookahead tool · a find tool

Q&A + polls on Slido →

THE PLAN

The elicitation ladder: one problem, one fix

Each rung fixes one problem of the baseline WikiAgent, and keeps the rungs below it. Measure each one.

PROBLEM IN THE BASELINE	RUNG	WHAT YOU CHANGE
Barely knows the goal; no strategy.	→ **1 · Prompting**	the three prompts

Q&A + polls on Slido →

THE PLAN

The elicitation ladder: one problem, one fix

Each rung fixes one problem of the baseline WikiAgent, and keeps the rungs below it. Measure each one.

PROBLEM IN THE BASELINE	RUNG	WHAT YOU CHANGE
Barely knows the goal; no strategy.	1 · Prompting	the three prompts
Acts without writing its reasoning.	2 · ReAct	a think call, then an act call

Q&A + polls on Slido →

THE PLAN

The elicitation ladder: one problem, one fix

Each rung fixes one problem of the baseline WikiAgent, and keeps the rungs below it. Measure each one.

PROBLEM IN THE BASELINE	RUNG	WHAT YOU CHANGE
Barely knows the goal; no strategy.	1 · Prompting	the three prompts
Acts without writing its reasoning.	2 · ReAct	a think call, then an act call
Forgets its plan after every move.	3 · Full history	keep the chat; drop old page text

Q&A + polls on Slido →

THE PLAN

The elicitation ladder: one problem, one fix

Each rung fixes one problem of the baseline WikiAgent, and keeps the rungs below it. Measure each one.

PROBLEM IN THE BASELINE	RUNG	WHAT YOU CHANGE
Barely knows the goal; no strategy.	1 · Prompting	the three prompts
Acts without writing its reasoning.	2 · ReAct	a think call, then an act call
Forgets its plan after every move.	3 · Full history	keep the chat; drop old page text
Finds broken routes only by walking them.	**4 · Lookahead**	a tool to test a path

Q&A + polls on Slido →

THE PLAN

The elicitation ladder: one problem, one fix

Each rung fixes one problem of the baseline WikiAgent, and keeps the rungs below it. Measure each one.

PROBLEM IN THE BASELINE	RUNG	WHAT YOU CHANGE
Barely knows the goal; no strategy.	→ **1 · Prompting**	the three prompts
Acts without writing its reasoning.	→ **2 · ReAct**	a think call, then an act call
Forgets its plan after every move.	→ **3 · Full history**	keep the chat; drop old page text
Finds broken routes only by walking them.	→ **4 · Lookahead**	a tool to test a path

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 4.1 why elicit

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

A failure is a floor

It says this scaffold didn't get there, not that the model can't.

absence ≠ evidence

2

Diagnose first

Bad prompt, bad history or bad tools? The failure tells you which fix to try.

prompt · history · tools

3

One rung at a time

Change one thing, measure it, keep it only if it helps.

the ladder

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

The WikiAgent makes a good plan, then forgets it after its next move. Which rung targets that?

- **A** Rewrite the three prompts' wording
- **B** Add a tool that tests whole routes
- **C** Keep history, drop old page text
- **D** Raise the message_limit to 200

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

4.2 Better prompts and ReAct

Questions on the last section? Ask in the room or on Slido.

THIS PART

4.1 Why elicit: every result is a floor ✓

4.2 Better prompts and ReAct

4.3 Memory and a lookahead tool

ASK NOW

Anything from the last section still unclear? This is the moment.

Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

WikiAgent sends three prompts. Rung 1 rewrites them.

These are the messages your scaffold writes. Notebook wording for the baseline WikiAgent.

system_instruction

the rules of the game

sent: start of every chat, and after each reset

"You are a wikipedia-racing AI. Your aim is to reach the goal page by accessing links from a series of wikipedia pages."

on_page_instruction

where you are, and the goal

sent: right after it; rebuilt after every move

"You are currently on page: Mango. Your goal page is Chandrayaan-3."

next_step_instruction

the nudge to act

sent: before every model call in the loop

"What will you do next?"

Q&A + polls on Slido →

WikiAgent sends three prompts. Rung 1 rewrites them.

These are the messages your scaffold writes. Notebook wording for the baseline WikiAgent.

system_instruction

the rules of the game

sent: start of every chat, and after each reset

"You are a wikipedia-racing AI. Your aim is to reach the goal page by accessing links from a series of wikipedia pages."

on_page_instruction

where you are, and the goal

sent: right after it; rebuilt after every move

"You are currently on page: Mango. Your goal page is Chandrayaan-3."

next_step_instruction

the nudge to act

sent: before every model call in the loop

"What will you do next?"

Q&A + polls on Slido →

WikiAgent sends three prompts. Rung 1 rewrites them.

These are the messages your scaffold writes. Notebook wording for the baseline WikiAgent.

system_instruction

the rules of the game

sent: start of every chat, and after each reset

"You are a wikipedia-racing AI. Your aim is to reach the goal page by accessing links from a series of wikipedia pages."

on_page_instruction

where you are, and the goal

sent: right after it; rebuilt after every move

"You are currently on page: Mango. Your goal page is Chandrayaan-3."

next_step_instruction

the nudge to act

sent: before every model call in the loop

"What will you do next?"

Q&A + polls on Slido →

WikiAgent sends three prompts. Rung 1 rewrites them.

These are the messages your scaffold writes. Notebook wording for the baseline WikiAgent.

system_instruction

the rules of the game

sent: start of every chat, and after each reset

"You are a wikipedia-racing AI. Your aim is to reach the goal page by accessing links from a series of wikipedia pages."

on_page_instruction

where you are, and the goal

sent: right after it; rebuilt after every move

"You are currently on page: Mango. Your goal page is Chandrayaan-3."

next_step_instruction

the nudge to act

sent: before every model call in the loop

"What will you do next?"

Q&A + polls on Slido →

Better prompts: same loop, better words

WikiAgent → WikiAgentPrompting. Only these three messages change. Notebook wording, shortened.

BEFORE · WikiAgent	AFTER · WikiAgentPrompting
system_instruction You are a wikipedia-racing AI. Your aim is to reach the goal page…	**system_instruction** …Your goal is to reach **Chandrayaan-3**… Your current page is Mango.
on_page_instruction You are currently on page: Mango. Your goal page is Chandrayaan-3.	**on_page_instruction** On page Mango. **Reason about your steps first.** Not working? **Try something else. Goal summary: […] Path so far: Mango.**
next_step_instruction What will you do next?	**next_step_instruction** What's your next step to reach Chandrayaan-3? **Think carefully.**

What changed: names the goal and the current page

Q&A + polls on Slido →

RUNG 1 · PROMPTING

Better prompts: same loop, better words

WikiAgent → WikiAgentPrompting. Only these three messages change. Notebook wording, shortened.

BEFORE · WikiAgent	AFTER · WikiAgentPrompting
system_instruction You are a wikipedia-racing AI. Your aim is to reach the goal page…	**system_instruction** …Your goal is to reach **Chandrayaan-3**… Your current page is Mango.
on_page_instruction You are currently on page: Mango. Your goal page is Chandrayaan-3.	**on_page_instruction** On page Mango. **Reason about your steps first.** Not working? **Try something else. Goal summary: […] Path so far: Mango.**
next_step_instruction What will you do next?	**next_step_instruction** What's your next step to reach Chandrayaan-3? **Think carefully.**

What changed: plan first · change strategy · goal summary · path so far

Q&A + polls on Slido →

Better prompts: same loop, better words

WikiAgent → WikiAgentPrompting. Only these three messages change. Notebook wording, shortened.

BEFORE · WikiAgent	AFTER · WikiAgentPrompting
system_instruction You are a wikipedia-racing AI. Your aim is to reach the goal page…	**system_instruction** …Your goal is to reach **Chandrayaan-3**… Your current page is Mango.
on_page_instruction You are currently on page: Mango. Your goal page is Chandrayaan-3.	**on_page_instruction** On page Mango. **Reason about your steps first.** Not working? **Try something else. Goal summary: […] Path so far: Mango.**
next_step_instruction What will you do next?	**next_step_instruction** What's your next step to reach Chandrayaan-3? **Think carefully.**

What changed: ties the nudge to the goal

Q&A + polls on Slido →

Better prompts: same loop, better words

WikiAgent → WikiAgentPrompting. Only these three messages change. Notebook wording, shortened.

BEFORE · WikiAgent	AFTER · WikiAgentPrompting
system_instruction You are a wikipedia-racing AI. Your aim is to reach the goal page…	**system_instruction** …Your goal is to reach **Chandrayaan-3**… Your current page is Mango.
on_page_instruction You are currently on page: Mango. Your goal page is Chandrayaan-3.	**on_page_instruction** On page Mango. **Reason about your steps first.** Not working? **Try something else. Goal summary: […] Path so far: Mango.**
next_step_instruction What will you do next?	**next_step_instruction** What's your next step to reach Chandrayaan-3? **Think carefully.**

Added: the goal by name · plan first · the goal's summary · the path so far. All rebuilt after every move.

Q&A + polls on Slido →

ReAct: Reason, then Act, then Observe

Yao et al. (2022). One step on Mango, read left to right. Two ordinary model calls; no new tool.

1

REASON

call 1 · tool_choice="none"

SCAFFOLD SENDS

"Before you decide on your next step, think carefully…"

Thought: Mango is about fruit. India is a hub; from there ISRO, then Chandrayaan-3.

Q&A + polls on Slido →

ReAct: Reason, then Act, then Observe

Yao et al. (2022). One step on Mango, read left to right. Two ordinary model calls; no new tool.

1 **REASON**
call 1 · tool_choice="none"

SCAFFOLD SENDS

"Before you decide on your next step, think carefully…"

Thought: Mango is about fruit. India is a hub; from there ISRO, then Chandrayaan-3.

2 **ACT**
call 2 · tools on

SCAFFOLD SENDS

"Now, based on your reasoning above, what…"

MovePageTool(page="India")

Q&A + polls on Slido →

ReAct: Reason, then Act, then Observe

Yao et al. (2022). One step on Mango, read left to right. Two ordinary model calls; no new tool.

1 REASON

call 1 · tool_choice="none"

SCAFFOLD SENDS

"Before you decide on your next step, think carefully…"

Thought: Mango is about fruit. India is a hub; from there ISRO, then Chandrayaan-3.

→

2 ACT

call 2 · tools on

SCAFFOLD SENDS

"Now, based on your reasoning above, what…"

MovePageTool(page="India")

→

3 OBSERVE

your code runs the tool

SCAFFOLD DOES

execute_tools() runs it on the board

tool: "Move successful"

THE TRACE BUILDS UP, ONE STEP AT A TIME

Thought 1: go to India

Thought 2: on India, find ISRO

Thought 3: on ISRO, click Chandrayaan-3

Q&A + polls on Slido →

ReAct in the loop, as pseudocode

Turning this into Inspect calls is your exercise. (Inspect also ships a ready-made react() agent.)

```
repeat until game.check_win():
    add "Think about your next step..."  # reason prompt
    thought = ask the model, tools OFF  # tool_choice="none"
    add the thought to the chat
    add "Now, based on your reasoning, act"  # act prompt
    reply = ask the model, tools ON
    add the reply; run any tool call
    moved? rebuild prompts, wipe chat  # as before
```

1 OF 2 · REASON

Tools visible, but switched off

tool_choice="none": the model sees the tools but can't call them.

So all it can write is a plan, which stays in the chat.

Q&A + polls on Slido →

ReAct in the loop, as pseudocode

Turning this into Inspect calls is your exercise. (Inspect also ships a ready-made react() agent.)

```
repeat until game.check_win():
    add "Think about your next step..."  # reason prompt
    thought = ask the model, tools OFF  # tool_choice="none"
    add the thought to the chat
    add "Now, based on your reasoning, act"  # act prompt
    reply = ask the model, tools ON
    add the reply; run any tool call
    moved? rebuild prompts, wipe chat  # as before
```

2 OF 2 · ACT

Same call as WikiAgent's

A new prompt, tools back on.

The action can now follow the plan it just wrote.

Note: the wipe after a move erases the plan too. Rung 3 fixes that.

Q&A + polls on Slido →

CHECKPOINT · END OF SECTION

Checkpoint: 4.2 prompts and ReAct

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

Three prompts

system, on-page, next-step. Rung 1 only rewrites their words.

same loop

2

ReAct = two calls

Think with tools off, then act with tools on. The plan sits in the chat.

tool_choice="none"

3

Still forgets

The wipe after a move erases the plan too. That's the next rung.

→ history

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

In ReAct, why does the reasoning call use tool_choice="none"?

A. So it writes a plan before it acts

B. To save tokens on tool descriptions

C. Because tools can't run twice a step

D. So Inspect doesn't log the thought

Vote on Slido

Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

4.3 Memory and a lookahead tool

Questions on the last section? Ask in the room or on Slido.

THIS PART

4.1 Why elicit: every result is a floor ✓

4.2 Better prompts and ReAct ✓

4.3 Memory and a lookahead tool

ASK NOW

Anything from the last section still unclear? This is the moment.

Scan to post a question on Slido. Upvote others' questions.

Q&A + polls on Slido →

RUNG 3 · HISTORY

What should the agent remember between pages?

After reading Mango and moving to India. Rough sizes: one page ≈ 4,000 tokens.

Reset every move

WikiAgent

system	rules + tools
user	On page: India. Goal: …

≈ 100 tokens

× the plan is gone

Q&A + polls on Slido →

What should the agent remember between pages?

After reading Mango and moving to India. Rough sizes: one page ≈ 4,000 tokens.

Reset every move

WikiAgent

system	rules + tools
user	On page: India. Goal: …

≈ 100 tokens

× the plan is gone

Keep everything

system	rules + tools
assistant	GetContentTool()
tool	The mango is… ≈ 4,000 tokens
assistant	MovePageTool("India")
tool	Move successful
assistant	GetContentTool()
tool	India is… ≈ 4,000 tokens

≈ 8,200 tokens, +4,000 a page

× fills the context window

Q&A + polls on Slido →

RUNG 3 · HISTORY

What should the agent remember between pages?

After reading Mango and moving to India. Rough sizes: one page ≈ 4,000 tokens.

Reset every move

WikiAgent

system	rules + tools
user	On page: India. Goal: …

≈ 100 tokens

✕ the plan is gone

Keep everything

system	rules + tools
assistant	GetContentTool()
tool	The mango is… ≈ 4,000 tokens
assistant	MovePageTool("India")
tool	Move successful
assistant	GetContentTool()
tool	India is… ≈ 4,000 tokens

≈ 8,200 tokens, +4,000 a page

✕ fills the context window

Keep history, drop old pages

WikiAgentHistory

system	rules + tools
assistant	GetContentTool()
tool	Mango content was output here…
assistant	MovePageTool("India")
tool	Move successful
assistant	GetContentTool()
tool	India is… ≈ 4,000 tokens

≈ 4,200 tokens

✓ plan kept, stays small

Q&A + polls on Slido →

LIVE DEMO · INTERACTIVE PAGE

Context Budget

Watch the chat's size across a whole game, for each history strategy, against gpt-4o-mini's 128k window.

OPEN

context_budget.html

Click the name to open. Also linked on the last slide. Any browser, nothing to install.

TRY THIS

1. Drag the move slider: when does “keep everything” run out?
2. Make pages longer (the US page is ~10k words).
3. Which strategy still remembers the plan at move 10?

Q&A + polls on Slido →

A lookahead tool: test the plan before walking it

Problem, idea, and how the notebook adapts Reflexion (Shinn et al., 2023).

1 Problem

It finds a broken hop only by walking into it.

Plan Mango → India → Chandrayaan-3: it moves to India, then gets “Move failed”.

2 Idea: Reflexion

Get **quick, cheap feedback** on a plan, and reflect on it, before acting.

In the paper an evaluator scores each try; the agent writes what went wrong into memory and tries again.

3 Adapted: a test tool

A third tool. The model proposes a **whole path**; it checks **every hop**.

Says success, or where it breaks. Shows **no page text**, and doesn't move the agent.

Q&A + polls on Slido →

A lookahead tool: test the plan before walking it

Problem, idea, and how the notebook adapts Reflexion (Shinn et al., 2023).

1 Problem

It finds a broken hop only by walking into it.

Plan Mango → India → Chandrayaan-3: it moves to India, then gets “Move failed”.

2 Idea: Reflexion

Get **quick, cheap feedback** on a plan, and reflect on it, before acting.

In the paper an evaluator scores each try; the agent writes what went wrong into memory and tries again.

3 Adapted: a test tool

A third tool. The model proposes a **whole path**; it checks **every hop**.

Says success, or where it breaks. Shows **no page text**, and doesn't move the agent.

Q&A + polls on Slido →

A lookahead tool: test the plan before walking it

Problem, idea, and how the notebook adapts Reflexion (Shinn et al., 2023).

1 Problem

It finds a broken hop only by walking into it.

Plan Mango → India → Chandrayaan-3: it moves to India, then gets “Move failed”.

2 Idea: Reflexion

Get **quick, cheap feedback** on a plan, and reflect on it, before acting.

In the paper an evaluator scores each try; the agent writes what went wrong into memory and tries again.

3 Adapted: a test tool

A third tool. The model proposes a **whole path**; it checks **every hop**.

Says success, or where it breaks. Shows **no page text**, and doesn't move the agent.

Q&A + polls on Slido →

A lookahead tool: test the plan before walking it

Problem, idea, and how the notebook adapts Reflexion (Shinn et al., 2023).

1 Problem

It finds a broken hop only by walking into it.

Plan Mango → India → Chandrayaan-3: it moves to India, then gets “Move failed”.

2 Idea: Reflexion

Get **quick, cheap feedback** on a plan, and reflect on it, before acting.

In the paper an evaluator scores each try; the agent writes what went wrong into memory and tries again.

3 Adapted: a test tool

A third tool. The model proposes a **whole path**; it checks **every hop**.

Says success, or where it breaks. Shows **no page text**, and doesn't move the agent.

Q&A + polls on Slido →

RUNG 4 · WHERE THE PLAN COMES FROM

Who plans, who checks?

The model writes the plan (as in ReAct's thought). The tool only checks it. The loop doesn't change.

REFLEXION, IN THE NOTEBOOK'S CHEAP VERSION

1 Actor

the model proposes

“Mango -> India -> Chandrayaan-3”

→

2 Evaluator

cheap feedback

TestPathTool checks each hop

→

3 Reflection

back in the chat

the tool message says where it broke

→

4 Next try

a better plan

the model re-plans, then walks it

vs ReAct

ReAct: the model thinks before each single move. Lookahead: it can test a whole multi-hop plan, cheaply, before moving at all.

```
tools = [GetContentTool(game),
         MovePageTool(game),
         TestPathTool(game)]   # only change
# notebook: WikiAgentHistory,
#           message_limit=120
```


Q&A + polls on Slido →

The lookahead tool, on our game

Pseudocode (writing it is your exercise). Feedback wording is the notebook's. Illustrative route.

```
# TestPathTool(path), as pseudocode
split path on "→" into page titles
if it doesn't start at the current page:
    return a failure message
for each hop  a → b:
    if b isn't a permitted link on a:
        return "works until b, not a link on a"
return "Success!"
# never moves the board
```

THE MODEL TRIES

Mango → India → Chandrayaan-3

THE TOOL SAYS

"This path works until Chandrayaan-3, which is not a permitted link on the page India."

THE MODEL RE-PLANS

Mango → India → Indian Space Research Organisation → Chandrayaan-3 → "Success!"

Q&A + polls on Slido →

The lookahead tool, on our game

Pseudocode (writing it is your exercise). Feedback wording is the notebook's. Illustrative route.

```
# TestPathTool(path), as pseudocode
split path on "→" into page titles
if it doesn't start at the current page:
    return a failure message
for each hop  a → b:
   if b isn't a permitted link on a:
      return "works until b, not a link on a"
return "Success!"
# never moves the board
```

THE MODEL TRIES

Mango → India → Chandrayaan-3

THE TOOL SAYS

“This path works until Chandrayaan-3, which is not a permitted link on the page India.”

THE MODEL RE-PLANS

Mango → India → Indian Space Research Organisation → Chandrayaan-3 → “Success!”

Q&A + polls on Slido →

The lookahead tool, on our game

Pseudocode (writing it is your exercise). Feedback wording is the notebook's. Illustrative route.

```
# TestPathTool(path), as pseudocode
split path on "→" into page titles
if it doesn't start at the current page:
    return a failure message
for each hop  a → b:
    if b isn't a permitted link on a:
        return "works until b, not a link on a"
return "Success!"
# never moves the board
```

THE MODEL TRIES

Mango → India → Chandrayaan-3

THE TOOL SAYS

“This path works until Chandrayaan-3, which is not a permitted link on the page India.”

THE MODEL RE-PLANS

Mango → India → Indian Space Research Organisation → Chandrayaan-3 → “Success!”

Q&A + polls on Slido →

RUNG 4 · WHAT HAPPENED

The lookahead tool mostly didn't help

From the notebook. A useful result in itself: adding a tool isn't free; the model has to use it well.

It drops a good path

“Mango -> India -> Chandrayaan-3” fails at the last hop.

Instead of adding one page (ISRO) and testing again, it abandons the route for a worse one.

It tests one step at a time

It asks “India -> Cricket”: a single hop.

It already sees India's links in the page from GetContentTool, so this tells it nothing new. The tool is meant for multi-hop routes it can't see.

A tool the model can't use well can make it worse. Measure each tool before you keep it.

Q&A + polls on Slido →

TWO RULES

Two things to keep in mind while eliciting

Which tricks are the safety worry, and how to tell if a change really helped.

General tricks are the worry

Narrow: helps one task, e.g. a tool that jumps to any page.

General: helps every task, e.g. “think step by step”.

One cheap general trick could raise every model's capability at once.

Compare fairly

Temperature 0: eval(..., temperature=0).

Pin the model version: gpt-4o-mini-2024-07-18.

Many runs, many pairs: utils.wiki_pairs has 18.

Q&A + polls on Slido →

RECAP · PAUSE

Part 4 recap

Three things to hold on to. Questions so far? Ask in the room or on Slido.

1

A failed eval is a floor

Before you write “can't”, check the prompts, the history and the tools.

absence ≠ evidence

2

One rung at a time

Prompts → ReAct → history → tools. Change one thing, measure, keep what helps.

measure each rung

3

General tricks are the worry

A trick like “think step by step” lifts every task at once, dangerous ones too.

compare fairly

Q&A + polls on Slido →

QUICK CHECK · SLIDO POLL

Your agent fails Joinery → Amethyst. What's the strongest honest claim?

- **A** The model can't play the Wikipedia game
- **B** ReAct would certainly have solved it
- **C** The two pages aren't connected at all
- **D** This scaffold didn't get the model there

Vote on Slido

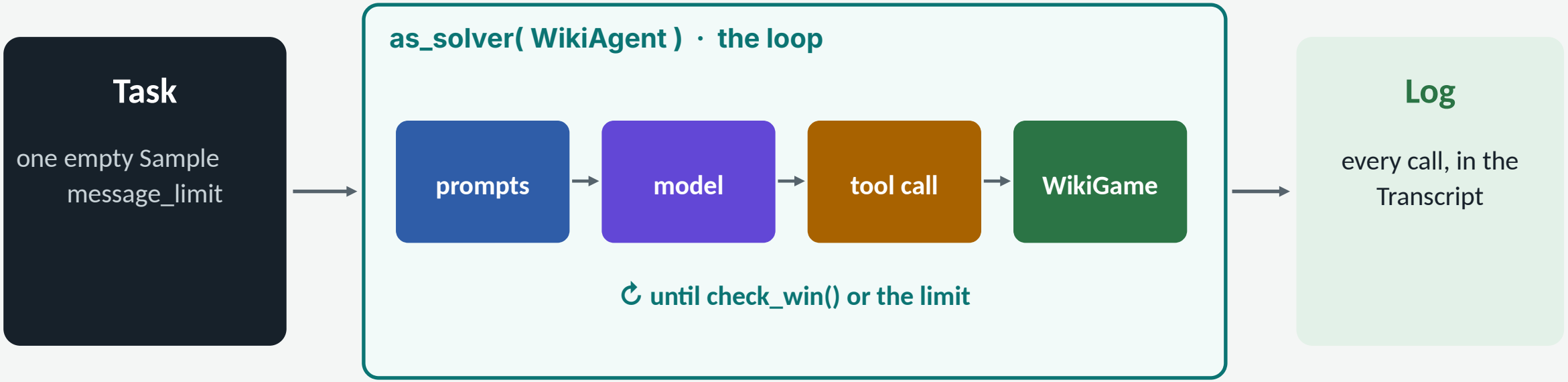
Scan to join. Pick one, then be ready to say why the others are wrong.

Q&A + polls on Slido →

WRAP-UP

The whole picture, on one slide

Everything today, in the order the eval runs it.



ELICITATION KNOBS · SAME MODEL, BETTER SYSTEM

prompts | ReAct | history | tools

An agent eval measures model + scaffold. Report what you tried, not just the score.

Q&A + polls on Slido →

TODAY'S NOTEBOOK

Notebook roadmap: what to do, in order

Aim to finish sections 1 and 2, then go as far as you can. Exercise help is in the appendix.

1 Intro

Read: what an agent is, why evaluate

- read only

2 Arithmetic agent

Your first agent and tool

- ArithmeticTask
- calculate tool
- arithmetic_agent
- run with eval()

3 WikiGame

Today's main build

- get_permitted_links
- GetContent + MovePage
- WikiAgent
- run the task

4 Elicitation

Climb the ladder

- better prompts
- ReAct
- full history
- TestPathTool

5 Bonus

Harder rules

- WikiGameRules
- your own ideas

Q&A + polls on Slido →

BEFORE PAIR PROGRAMMING

Five things that will save you an hour

The snags people hit most often in this notebook.

Limit every loop	Set `message_limit` on the Task, always.
Model name in part 4	OpenRouter key only? Use `openrouter/openai/gpt-4o-mini-2024-07-18`.
Compare at temperature 0	`eval(..., temperature=0)`
Read the transcript	Did it win, go in circles, or hit the limit?
Print the history after each move	Most WikiAgent bugs: a stale prompt, or a reset at the wrong time.

Q&A + polls on Slido →

FURTHER READING

If you want to go deeper

In the TARA pre-reading's priority order. If you read only one thing, read Weng.

LLM Powered Autonomous Agents Lilian Weng, 2023	*~20 min · the map: planning, memory, tool use*
Evaluating Language-Model Agents on Realistic Autonomous Tasks Kinniment et al. (METR), 2023	*~10 min · intro and section 2*
Inspect docs: Agents inspect.aisi.org.uk	*~15 min · before Part 3*
ReAct: Synergizing Reasoning and Acting in Language Models Yao et al., 2022	*skim · abstract + Figure 1*
Reflexion: Language Agents with Verbal Reinforcement Learning Shinn et al., 2023	*skim · before the lookahead exercise*
LLMs can Strategically Deceive their Users when Put Under Pressure Scheurer et al. (Apollo Research), 2023	*skim · a real agent eval*
AXRP Episode 34: AI Evaluations with Beth Barnes Daniel Filan, 2024	*optional · podcast*

Q&A + polls on Slido →

THANK YOU

Questions? Then on to the notebook

DEMOS

Log Viewer	Part 3
Context Budget	Part 4
Tool Schema Lab	Part 3

Notebook: ARENA 3.4 LLM Agents (exercises). Solutions are in the collapsed cells; try first.

Q&A + polls on Slido →

APPENDIX

Exercise help, for pair programming

Not presented. Hints, pseudocode and Python helpers. No solutions.



APPENDIX A1 · COVERAGE

Every notebook section, and where it was covered

Slide numbers in this deck.

§1	What an agent is; why evaluate agents	3, 17–21, 41
§2	ArithmeticTask; the loop on the arithmetic task	13, 28
§2	Tools: @tool, docstring → description	69–73
§2	Agents: @agent, AgentState, as_solver, eval	87–90, 95–108
§3	Wikipedia API; permitted links; page names	63–65
§3	WikiGame, the two tools	48–62
§3	WikiAgent: prompts, loop, reset	112–122
§4	Prompting, ReAct	140–153
§4	Full history; reflexion (lookahead) tool	156–159, 161–169
§5	Bonus: rules	174

Q&A + polls on Slido →

Inspect cheat sheet: the API names in this notebook

Everything new this week, in one place.

Tools

@tool · async def execute(arg: type) → str
docstring with an Args: line per argument
execute_tools(messages, tools)
  → tool messages to add to the chat

Agents

@agent · async def execute(state: AgentState)
state.messages · state.output
get_model().generate(input=, tools=,
  tool_choice="auto" | "none")

Messages

ChatMessageSystem(content=...)
ChatMessageUser(content=...)
ChatMessageTool (.function, .content)
output.message.tool_calls
→ .function, .arguments

Running

Task(dataset=[Sample(input="", target="")],
    message_limit=80)
eval(tasks=..., solver=as_solver(agent))
inspect view → the log viewer

Wikipedia

wikipedia.page(title, auto_suggest=False,
          redirect=True)
page.title · page.content · page.links
PageError · DisambiguationError

Q&A + polls on Slido →

Exercises: notebook part 2, the arithmetic agent

Do these in pairs, in order. The right-hand column tells you how to check you're done.

EXERCISE	RATING	TIME	YOU'RE DONE WHEN…
ArithmeticTask	●●●○○ ●●○○○	20–25 min **if time**	tests.test_arithmetic_task passes; prints 3 + 5, then 3 - 5.
calculate tool	●●○○○ ●●●●○	10–15 min	tests.test_calculate_tool passes. Docstring has an Args: line.
arithmetic_agent	●●●●○ ●●●●●	20–25 min	Runs all 7 problems; tool calls and tool messages appear in order.
run with eval()	●○○○○ ●●●○	5–10 min	A log opens; it ended by solving, not by hitting message_limit.

● difficulty ● importance · times are ARENA's estimates · **if time** = lower priority when short on time (TARA pre-reading)

Q&A + polls on Slido →

THE WHOLE AGENT

The arithmetic agent as pseudocode

Plain-English steps. Turning these into Inspect calls is your exercise.

```
# the conversation starts with the empty message
repeat:
    add the current sum     # from ArithmeticTask
    reply = ask the model   # tools allowed
    if the reply asks for the calculator:
        run it, add the result
    reply = ask the model   # text only
    if the answer is right:
        move to the next sum
    else:
        add "Incorrect answer. Try again."
until all 7 are right
```

HOW THE LOOP ENDS



Goal reached

Your scaffold compares each answer with ArithmeticTask's. All 7 right: stop.



Limit hit

message_limit on the Task. Inspect ends the run; the log says “Message limit exceeded”. Always set one.

Q&A + polls on Slido →

Notebook part 2: the arithmetic agent

Where to look, and what trips people up. No code: the solutions are in the notebook's collapsed cells.

EXERCISE	WHERE TO LOOK	WATCH OUT / CHECK
ArithmeticTask	the 7 operations; answers via evaluate_expression (utils)	get_current_task is a property: no ( ). Answers are strings like "8.0"
calculate	the toy @tool example; evaluate_expression from utils	never eval(). Every argument needs an Args: line
arithmetic_agent	the pseudocode slide; extract_answer reads <ANSWER> tags	append each reply yourself; move on only after a right answer
run with eval()	agent_task() is given; wrap the agent in as_solver	always set message_limit, or a stuck loop never ends

Q&A + polls on Slido →

Exercises: notebook part 3, the WikiAgent

Do these in pairs, in order. The right-hand column tells you how to check you're done.

EXERCISE	RATING	TIME	YOU'RE DONE WHEN…
get_permitted_links	●○○○○ ●●○○○	~10 min **if time**	tests.test_get_permitted_links passes; about a third of the links remain.
GetContent + MovePage	●●○○○ ●●●○○	15–20 min	Content shows <link> tags; a bad move returns the failure message.
WikiAgent	●●●●○ ●●●●●	30–60 min	Python (programming language) → Artificial intelligence reaches the goal.
run the task	●●○○○ ●●○○○	10–15 min **if time**	Elizabeth I → United States and County Seat → Saint Pierre and Miquelon. Rerun once if it fails.

● difficulty ● importance · times are ARENA's estimates · **if time** = lower priority when short on time (TARA pre-reading)

Q&A + polls on Slido →

APPENDIX A4 · EXERCISE HINTS (NO CODE)

Notebook part 3: the WikiGame and WikiAgent

Where to look, and what trips people up. No code: the solutions are in the notebook's collapsed cells.

EXERCISE	WHERE TO LOOK	WATCH OUT / CHECK
get_permitted_links	page.links (every link) vs page.content (main text)	compare lowercase to lowercase; drop the page's own title
GetContentTool	the regex hint wraps each permitted link in <link> tags	wrap longest titles first (the hint sorts by length)
MovePageTool	game.is_permitted_link, game.get_page	update current_page AND page_history; return the exact success / failure text
WikiAgent prompts	f-strings using game.current_page.title and goal_page.title	f-strings fill in once: rebuild them after a move (nonlocal)
_handle_tool_calls	execute_tools(messages=..., tools=...)	reset only after a SUCCESSFUL move; the name is "MovePageTool", not "execute"
run the task	WikiGame(start, goal); the same game in both tools	Elizabeth I → United States should win

Q&A + polls on Slido →

Notebook part 3: the pieces, as pseudocode

Structure only. Turning each line into code is the exercise.

notebook §3 · board and tools

```
# get_permitted_links()
for each link on the current page:
    keep it if it appears in the main text
        (compare in lowercase)
drop the current page's own title
return the kept links

# MovePageTool.execute(page)
if page is a permitted link:
    move the board, add to page_history
    return a success message
else:
    return a failure message
```

notebook §3 · WikiAgent helpers

```
# _start(state)
add system_instruction, on_page_instruction

# _handle_tool_calls(state)
run the tool calls, add their results
if it was MovePageTool and it succeeded:
    instruction_refresh()
    _reset_history(state)

# instruction_refresh()
rebuild the three prompts from game
    (nonlocal: so the loop sees them)
```


Q&A + polls on Slido →

Exercises: notebook part 4, elicitation

Do these in pairs, in order. The right-hand column tells you how to check you're done.

EXERCISE	RATING	TIME	YOU'RE DONE WHEN…
Engineer prompts	●●○○○ ●●●○○	20–35 min	Mandate of Heaven → Doric Greek at temperature 0, more reliably than WikiAgent.
WikiAgentReAct	●●○○○ ●●●○○	15–20 min	The log shows a text-only reasoning reply before each action.
WikiAgentHistory	●●○○○ ●●○○○	10–15 min	Old page text becomes the placeholder; plans stay in the history.
TestPathTool	●●●○○ ●●●○○	25–35 min **if time**	Bad paths return “This path works until…”. Compare with and without it.

● difficulty ● importance · times are ARENA's estimates · **if time** = lower priority when short on time (TARA pre-reading)

Q&A + polls on Slido →

Notebook part 4: elicitation, and the bonus

Where to look, and what trips people up. No code: the solutions are in the notebook's collapsed cells.

EXERCISE	WHERE TO LOOK	WATCH OUT / CHECK
Engineer prompts	game.get_page_summary(goal_page); join page_history with " -> "	rebuild after every move; keep prompts general, not tuned to one pair
WikiAgentReAct	a generate call with tool_choice="none", then a normal one	save both replies; run tools only after the act call
WikiAgentHistory	ChatMessageTool messages whose .function is "GetContentTool"	don't wipe: replace old page text with the “…removed for brevity” note
TestPathTool	split the path on "->"; check each hop's permitted links	the path must start at the current page; never move the board
Bonus rules	WikiGameRules(rules=[...]); page.categories for “no countries”	enforce in MovePageTool (and TestPathTool); tell the model in the prompt

Q&A + polls on Slido →

Notebook part 4: the new pieces, as pseudocode

Structure only. Copy your earlier solutions for everything else.

notebook §4 · ReAct and history

```
# WikiAgentReAct: one round
add the reason prompt
ask the model with tools OFF
add its thought
add the act prompt
ask the model with tools ON
add the reply; handle tool calls
check the win

# WikiAgentHistory._reset_history
for each old GetContentTool result:
    swap its text for the short note
keep every other message
```

notebook §4 · the lookahead tool

```
# TestPathTool.execute(path)
split the path into page titles
if it doesn't start at the current page:
    return a failure message
for each hop (a → b):
    if b isn't a permitted link on a:
        return where it breaks
return success

# never move the board here
```


Q&A + polls on Slido →

The Python bits the exercises use

Not solutions: the language and API pieces you'll need.

Rebind a variable from an inner function

nonlocal

```
def WikiAgent(...):
    prompt = make_prompt()
    async def refresh():
        nonlocal prompt
        prompt = make_prompt()
```

f-strings are filled in once

why refresh exists

```
p = f"on page: {game.current_page.title}"
# game moves to India...
# p still says Mango
```

Compare titles safely

lowercase both

```
link.lower() in content.lower()
```

Join a path

for prompts

```
" → ".join(["Mango", "India"])
# 'Mango → India'
```

Find old page reads

isinstance

```
isinstance(m, ChatMessageTool)
    and m.function == "GetContentTool"
```

Read the tool calls

on the last reply

```
for call in state.output.message.tool_calls:
    call.function, call.arguments
```


Q&A + polls on Slido →

When it breaks: what you'll see, and the usual cause

Read the log's Transcript first. Most bugs are visible there.

Message limit exceeded	the win check never comes true, or the loop never stops
ValueError: Description not provided for parameter	a tool's docstring is missing an Args: line
PageError / DisambiguationError	a title that isn't exact; get_page handles it with redirect, auto_suggest and the first option
coroutine … was never awaited	a missing await on generate(), execute_tools() or your agent
The prompt still names the old page	the prompts weren't rebuilt after the move (f-string filled once)
It retries the same bad link	you reset after a FAILED move, so the failure message was erased
Context length exceeded	whole pages are kept in the history

Q&A + polls on Slido →